

Claude Code 토큰 소비 절반으로 줄이기

1200시간 운영 데이터에서 발견한 실전 테크닉

저자: 김재우 jaewoo@mdrules.dev

이 PDF문서는 삼성전자 반도체부분 CLAUDE CODE 교육에 참고자료로 제공된 문서로 외부유출을 금지합니다.

1. 시작하며 — 왜 토큰이 사라지는가

버전 정보: 이 내용은 2026년 4월 기준 Claude Code(v2.1.x)에 기반합니다.

Anthropic 업데이트에 따라 토큰 소비 구조가 변경될 수 있으나, CLAUDE.md 작성법·워크플로우 설계 원칙은 버전에 크게 의존하지 않습니다.

Claude Code를 쓰기 시작하면서 가장 먼저 마주친 문제가 바로 토큰 소비 속도였다.

Max 플랜(월 \$200)에 가입하면 하루 종일 쓸 수 있을 거라 생각했다. 그런데 실제로는 아침부터 사용하기 시작하면 점심쯤엔 쿼터의 절반이 사라져 있었다.

"아무것도 안 했는데 토큰이 줄어든다." GitHub Issue의 토큰 소비 스레드에는 2,500건 이상의 리액션이 달려 있다. Claude Code 사용자들이 공통으로 겪는 가장 큰 고충이다.

<https://github.com/anthropics/claude-code/issues/42796>

이 책을 쓴 이유

1200시간 이상 Claude Code를 자율 운영했다. 그 과정에서 같은 작업을 해도 토큰 소비량이 크게 달라지는 패턴이 있다는 사실을 발견했다.

- **CLAUDE.md** 작성법을 바꾸기만 했는데, 세션당 소비량이 눈에 띄게 줄었다
- **컨텍스트 관리** 방식을 바꿨더니, `/compact` 실행 빈도가 절반으로 떨어졌다
- **훅(Hook)**을 활용해 토큰 낭비 패턴을 자동으로 감지·차단할 수 있게 되었다
- **워크플로우 설계**를 바꿨더니, 동일한 결과물을 더 적은 토큰으로 만들어낼 수 있었다

이 책은 그 시행착오의 기록이다.

"Claude Code한테 직접 물어보면 안 되나요?"

자연스러운 발상이다. "토큰을 절약하려면 어떻게 해?"라고 Claude Code에 물어보면 그만 아닌가 싶기도 하다.

그런데 실제로는 잘 통하지 않는다.

Claude Code에 "토큰을 절약해줘"라고 입력하면, 그 지시가 CLAUDE.md나 lessons.md에 기록된다. Claude Code는 세션 시작 시 해당 파일을 불러온다. 그러면 어떤 일이 벌어지는가 — **토큰을 많이 쓸 것 같은 작업을 스스로 피하기 시작한다.**

구체적으로는 이런 식이다:

- 대용량 파일 생성 작업을 자체적으로 회피한다
- 상세한 분석·조사를 생략하고 짧은 답변으로 때우려 한다
- 서브에이전트 활용을 자제한다 (서브에이전트는 토큰을 많이 소모하기 때문에)
- 결과적으로 **산출물의 품질이 저하된다**

"토큰 절약"이라는 말이 지시 파일에 들어가는 순간, Claude Code는 성과 극대화가 아닌 소비 최소화를 최우선으로 삼게 된다. 이는 본말이 전도된 접근이다.

올바른 토큰 절약이란, Claude Code의 행동을 위축시키는 것이 아니다. 동일한 품질의 결과물을 더 적은 토큰으로 달성하는 것이다. 이를 위해서는 "절약해줘"라고 부탁하는 대신, CLAUDE.md의 구조·훅(Hook)에 의한 자동 제어·워크플로우 설계라는 **구조적 수준**에서 대응해야 한다.

이 책은 바로 그 구조를 체계화한 것이다.

누구를 위한 책인가

- Claude Code를 사용 중이며 "토큰 소비가 너무 빠르다"고 느끼는 사람
- Max 플랜 월 \$200을 제대로 활용하고 싶은 사람
- 자율 운영(방치해두고 알아서 작업하게 하는 방식)에서 토큰 효율을 높이고 싶은 사람
- "훅이나 CLAUDE.md로 토큰을 절약할 수 있다"고는 들었지만, 구체적인 방법을 모르는 사람

기술적 사전 지식은 필요 없다. 훅 코드는 모두 복사·붙여넣기로 바로 쓸 수 있게 정리해두었다. 커스터마이징을 원한다면 셸 스크립트 기초 지식이 있으면 편리하지만, 필수는 아니다.

이 책의 구성

각 장은 독립적으로 구성되어 있으니, 관심 있는 장부터 읽어도 된다.

장	제목	핵심 내용
2장	토큰은 어디서 소비되는가?	소비 메커니즘 전체 구조. 무엇이 토큰을 먹는가
3장	CLAUDE.md 최적화	지시 작성법에 따라 소비가 달라지는 이유와 실증된 5가지 패턴
4장	컨텍스트 관리	<code>/clear</code> , <code>/compact</code> , <code>--resume</code> 의 올바른 사용법. 흔한 실수
5장	훅(Hook)으로 토큰 낭비 자동 차단	불필요한 동작을 감지·차단하는 9가지 훅
6장	서브에이전트 전략	메인 컨텍스트를 오염시키지 않고 병렬 처리하는 방법
7장	워크플로우 설계	작업 분할과 순서 조정으로 토큰 효율 높이기
8장	1200시간 소비 데이터	실제 소비 패턴 공개 및 분석
9장	트러블슈팅	캐시 깨짐, 쿼터 급증, 자주 발생하는 문제와 대처법
부록	설정 템플릿 모음	CLAUDE.md · hooks · settings.json 복붙 전용 템플릿

제1장 토큰은 어디서 사라지는가?

버전 정보: 이 내용은 2026년 4월 기준 Claude Code(v2.1.x)에 기반합니다.

"아무것도 안 했는데 줄어든다" — Claude Code 사용자들이 가장 많이 하는 불만이다. 하지만 정말로 "아무것도 안 한" 것은 아니다. 보이지 않는 곳에서 토큰이 소비되고 있을 뿐이다.

이 장에서는 토큰이 어디서 쓰이는지 전체 구조를 정리한다. 원인을 파악하면 대책이 보인다.

토큰 소비의 4가지 계층

Claude Code의 토큰 소비는 크게 4가지 계층으로 나뉜다.

계층 1: 시스템 프롬프트 (세션 시작 시)

Claude Code가 실행될 때마다 내부 시스템 프롬프트가 불러와진다. 사용자 눈에는 보이지 않지만 매번 토큰을 소비한다.

세션 시작 시 함께 불러오는 항목은 다음과 같다:

- **CLAUDE.md**: 글로벌(`~/.claude/CLAUDE.md`)과 프로젝트 로컬(`./CLAUDE.md`) 양쪽 모두
- **툴 정의**: 사용 가능한 툴 설명. Deferred Tool Loading(v2.1.89 이후)으로 일부는 지연 로드되지만, 기본 툴 묶음은 항상 포함된다
- **MCP 설정**: MCP 서버의 툴 정의도 추가된다

즉, CLAUDE.md가 길수록 세션 시작 시 토큰 소비가 늘어난다. 100줄짜리와 500줄짜리는 매 턴 기본 소비량 자체가 달라진다.

계층 2: 컨텍스트 윈도우 (대화 기록의 누적)

대화가 진행될수록 과거 대화 기록 전체가 컨텍스트 윈도우에 쌓인다. Claude Code는 1M 토큰의 컨텍스트를 지원하지만, 누적량이 많아질수록:

- 매 턴마다 전송되는 토큰 양이 증가한다
- 응답 생성에 쓰이는 토큰도 함께 늘어난다
- 프롬프트 캐시가 유효하면 재사용되지만, 캐시가 깨지면 전체 토큰이 다시 카운트된다

프롬프트 캐시 무효화는 토큰 급증의 가장 큰 원인 중 하나다 (제8장에서 상세 설명).

계층 3: 툴 호출 (읽기·쓰기 실행)

Claude Code가 파일을 읽거나 Bash 커맨드를 실행할 때마다 토큰이 소비된다:

툴	소비 방식
Read	파일 내용이 컨텍스트에 추가됨. 파일이 클수록 소비 대폭 증가
Glob / Grep	검색 결과가 컨텍스트에 포함됨
Bash	커맨드 출력이 컨텍스트에 포함됨. <code>ls -la</code> 와 <code>cat large-file.txt</code> 의 소비량 차이는 수십 배
Edit / Write	변경 내용이 컨텍스트에 기록됨

⚠️ 추가된 내용: 원문의 목록을 한국 독자 가독성을 위해 표 형식으로 재구성했습니다.

툴 호출 시마다 입력(툴 파라미터)과 출력(실행 결과) 양쪽 모두 토큰으로 카운트된다.

또한 툴 결과에는 총 **200K 토큰**의 내부 예산 상한선이 있다. 이를 초과하면 툴 결과가 사용자 동의 없이 잘려나간다. 그러면 불완전한 정보를 기반으로 Claude가 판단 → 재시도 → 추가 소비하는 악순환이 발생한다 (제8장에서 상세 설명).

계층 4: 서브에이전트 (병렬 처리의 비용)

Agent 툴로 서브에이전트를 실행하면 각 서브에이전트가 독자적인 컨텍스트를 갖는다. 메인 컨텍스트에서는 내용이 사라지지 않지만, API 호출 횟수는 증가한다.

서브에이전트는 강력한 도구지만 남용하면 토큰 소비가 급증한다 (제5장에서 상세 설명).

먼저 내 소비량을 확인하자

대책을 세우기 전에 현재 소비량을 파악하는 것이 우선이다. Claude Code에는 두 가지 확인 방법이 있다:

- `/cost`: API 종량제 사용자용. 세션 중 토큰 소비량과 비용을 표시
- `/stats`: Max/Pro 플랜 사용자용. 사용 패턴을 표시 (구독 과금 방식이므로 금액은 표시되지 않음)

상태 라인에 상시 표시하는 것도 가능하다 (`/config` → statusline 설정). 최적화 전후로 `/cost` 값을 비교하는 습관을 들여야 대책의 효과를 제대로 측정할 수 있다.

무엇이 토큰을 가장 많이 먹는가

1200시간 운영 데이터를 기반으로 토큰 소비량이 큰 순서대로 정리하면:

순 위	소비 원인	내용
 1	대용량 파일 읽기	수천 줄짜리 파일을 Read로 불러오면 한 번에 대량 소비
 2	컨텍스트 비대화	<code>/compact</code> 없이 장시간 사용 시 매 턴 기본 소비량이 팽창
 3	프롬프트 캐시 무효 화	캐시가 깨지면 같은 프롬프트라도 소비가 수 배로 증가
4	불필요한 톨 호출	파일 목록 조회, git 로그 출력 등 불필요한 정보 수집
5	CLAUDE.md 비대 화	500줄을 초과하는 CLAUDE.md는 매 턴 수천 토큰의 고정 비용 발생
6	상태 관리 파일 빈번 한 업데이트	<code>mission.md</code> · <code>task-queue</code> 를 톨 호출마다 갱신하는 패턴 (대책과 실 측 데이터는 제7장에서 상세 설명)

제2장 CLAUDE.md 작성법에 따라 토큰 소비가 달라진다

CLAUDE.md는 Claude Code에 대한 지시 파일이다. 여기에 작성한 내용은 세션 중 모든 턴의 컨텍스트에 포함된다. 즉, CLAUDE.md가 1줄 늘어날 때마다 세션 전체의 토큰 소비가 증가한다.

그러나 "짧게 쓰면 된다"는 단순한 이야기가 아니다. 너무 짧은 지시는 모호해지고, Claude Code가 의도를 파악하지 못해 재시도를 반복한다. 그 재시도가 오히려 훨씬 더 많은 토큰을 소비한다.

중요한 것은 "짧은"이 아니라 **"효율적인"** 것이다. 동일한 의도를 더 적은 토큰으로, 더 정확하게 전달하는 작성법이 있다.

원칙: CLAUDE.md는 100줄이 기준

1200시간 운영에서 얻은 경험칙으로, CLAUDE.md는 **100줄 이내**가 효율적이다. Anthropic 공식 문서에서도 "간결하게"라고 권장하고 있지만, 구체적인 줄 수는 제시되어 있지 않다. 아래는 운영 실적에 기반한 분류다.

줄 수	평가
50줄 이하	기본 지시만 들어간다. 자율 운영에는 부족한 경우가 많다
50~100줄	주요 규칙과 판단 기준이 들어간다. 가장 균형이 좋다
100~200줄	세부 규칙까지 커버할 수 있지만 고정 토큰 비용이 증가하기 시작한다
200줄 이상	매 턴 오버헤드가 체감할 수 있는 수준. 분리를 검토해야 한다

200줄 이상이 되면 `.claude/rules/` 디렉토리로 분리를 검토한다. Claude Code v2.1.x 계열에서는 `.claude/rules/` 내의 Markdown 파일도 시스템 프롬프트에 포함되지만, CLAUDE.md와 달리 파일 단위로 관리할 수 있어 **"항상 읽는 지시"**와 **"특정 조건에서 읽는 지시"**를 분리할 수 있다. 주요 판단 기준과 우선순위는 CLAUDE.md에 남기고, 상세한 절차나 조건 분기는 `.claude/rules/`로 옮기는 것이 실용적인 패턴이다.

저자의 CLAUDE.md는 최종적으로 141줄에 정착했다. 여기에 이르기까지 500줄 이상이었던 시기도 있다. 제9장 템플릿에서는 30줄 정도의 최소 구성도 소개한다. 프로젝트 복잡도에 따라 **30~100줄** 사이에서 조정하는 것이 현실적이다.

패턴 1: 금지 목록보다 허용 목록

✗ 비효율적인 작성법

금지 사항

- `rm -rf`를 사용하지 말 것
- `git reset --hard`를 사용하지 말 것
- `git push --force`를 사용하지 말 것
- `.env` 파일을 삭제하지 말 것
- `node_modules`를 삭제하지 말 것
- `/`를 포함한 경로에서 `rm`을 사용하지 말 것
- `sudo`를 사용하지 말 것

7줄로 7개의 금지 사항. 그러나 "금지되지 않은 작업은 허용"으로 해석될 여지가 있다. `find . -delete`는 금지되지 않았으니 사용 가능하다고 판단할 수 있다. 금지 목록은 항상 불완전하다.

✅ 효율적인 작성법

허용 작업

- 파일 읽기: 항상 가능
- `git commit`: 가능 (main으로 직접 push는 불가)
- `npm install`: 가능 (`--save-dev`만)
- 파일 삭제: 자신이 생성한 파일만

위에 없는 파괴적인 작업은 실행 전에 확인.

6줄로 같은 범위를 커버하면서, 목록에 없는 작업에 대해 신중하게 동작한다. 화이트리스트(허용 목록) 방식은 보안의 기본 원칙과 동일하다.

토큰 절약 효과: 줄 수는 비슷하지만 재시도 감소 효과가 크다. 금지 목록 방식에서는 "이게 금지된 것인가?"라는 판단이 매번 발생하고, 판단을 잘못하면 "혹에 차단됨 → 다른 방법 시도 → 또 차단"의 루프가 된다. 허용 목록 방식에서는 "목록에 없으니 하지 않는다"로 한 번에 끝난다.

패턴 2: 구체적인 예시를 1개 붙인다 (퓨샷, Few-shot)

❌ 비효율적인 작성법

커밋 메시지는 명확하게 작성할 것.

Claude Code가 이 지시로 생성하는 커밋 메시지: `Update files` `Fix bug` `Add feature`. 명확한지 아닌지의 기준이 없기 때문이다.

✅ 효율적인 작성법

커밋 메시지: "왜"를 포함한다.

- ✓ `fix: prevent duplicate API calls on rapid button clicks`
- ✗ `fix bug`

3줄 추가만으로 품질이 극적으로 달라진다. LLM은 구체적인 예시에서 패턴을 추출하는 것을 잘한다(퓨샷 프롬프팅). 좋은 예 1개와 나쁜 예 1개가 있으면, 그 사이의 판단 기준을 스스로 보완한다.

토큰 절약 효과: 예시를 2줄 추가하는 비용(매 턴 +수십 토큰)에 비해, "커밋 메시지 부적절 → 재작성 요청 → 재 생성"의 재시도(수백 토큰)가 사라진다. 투자 대비 효과가 높다.

패턴 3: 이유를 1줄만 붙인다

✗ 비효율적인 작성법

테스트 파일은 ``__tests__``에 배치한다.

이유가 없는 규칙은 예외 케이스에서 무시되기 쉽다. "src/ 옆에 두는 것이 더 명확하겠는데"라고 Claude Code가 판단해 규칙을 어기는 경우가 있다.

✓ 효율적인 작성법

테스트 파일은 ``__tests__``에 배치한다.
이유: `src/`와 같은 계층에 두면 빌드 출력에 혼입되어 번들 크기가 증가한다

1줄의 이유를 추가하는 것만으로 Claude Code는 "왜 안 되는지"를 이해한다. 유사한 케이스(예: 설정 파일 배치)에도 자발적으로 규칙을 적용하게 된다.

주의: 이유는 **1~2줄로 충분**하다. 3단락 이상의 설명은 역효과다. 운영 데이터에서 긴 이유 설명이 토큰 소비를 체감 약 10% 전후 증가시켰다. 1줄의 이유라면 거의 오차 범위 수준이다. 구체적인 수치 비교는 제7장에서 설명한다.

패턴 4: 테이블 형식으로 판단 기준을 작성한다

✗ 비효율적인 작성법

기술 선택에서 망설이면, 가장 대중적인 옵션을 선택해 주세요.
요구사항이 불명확한 경우는, 가장 단순한 해석으로 구현해 주세요.
성능과 가독성이 충돌하면, 보통은 가독성을 우선해 주세요.
단, 프로파일링으로 병목이 확인된 경우는 성능을 우선해 주세요.
보안에 관한 판단은, 항상 안전한 쪽을 선택해 주세요.

5줄의 일반 텍스트. Claude Code는 매번 이 문장을 전부 읽고 해당하는 부분을 찾아야 한다.

✓ 효율적인 작성법

상황 판단
----- -----
기술 선택 가장 대중적인 것
불명확한 요구사항 가장 단순한 해석
성능 vs 가독성 가독성 우선 (프로파일링 완료 시 예외)
보안 항상 안전한 쪽

동일한 정보량으로 구조가 명확하다. LLM은 테이블 형식의 조건 분기를 높은 정확도로 처리한다. 일반 텍스트보다 파싱이 빠르고 오독이 적다.

토큰 절약 효과: 테이블 형식은 일반 텍스트보다 적은 토큰으로 동일한 정보량을 표현할 수 있는 경향이 있다(구조화로 Claude 측 해석 비용도 낮아진다). 게다가 판단 실수로 인한 재시도가 줄어든다.

패턴 5: 훅(Hook)에 맡겨야 할 규칙을 CLAUDE.md에서 제거한다

❌ 비효율적인 작성법

안전 규칙

- `rm -rf`는 절대 사용하지 않는다. 대신 파일을 개별 삭제한다
- `git reset --hard`는 사용하지 않는다. 실수한 경우 `git reflog`로 복구한다
- `.env` 파일은 `git add` 하지 않는다. `.gitignore`에 포함되어 있는지 확인한다
- `main` 브랜치에 직접 `push`하지 않는다. `feature` 브랜치를 만든다
- `sudo`는 사용하지 않는다
- `chmod 777`은 사용하지 않는다

이것들은 CLAUDE.md에 적어도 "부탁"에 불과하다. Claude Code가 규칙을 잊을 가능성은 0이 아니다.

✅ 효율적인 방법

훅으로 강제하고, CLAUDE.md에는 적지 않는다:

안전 규칙

훅(Hook)이 파괴적 커맨드·브랜치 보호·시크릿 감지를 자동 차단한다.
훅이 차단한 작업은 재시도하지 말 것.

2줄로 충분하다. 훅이 기술적으로 강제하는 규칙을 CLAUDE.md에도 중복 작성하는 것은 이중 관리다. 토큰 낭비이자 관리 부담이기도 하다.

토큰 절약 효과: 위의 예에서 6줄 → 2줄로 4줄분의 고정 토큰 비용 절감. 게다가 훅이 차단하기 때문에 재시도 사이클이 "시도 → 차단 → 포기"의 1회로 끝난다. CLAUDE.md만 사용하는 경우는 "시도 → 성공해버림 → 사고 → 롤백"으로 수 배의 토큰을 낭비하게 된다.

정리: 5패턴 빠른 참조표

패턴	핵심	절약 메커니즘
허용 목록	"해도 되는 것"을 작성한다	재시도 감소
구체적인 예시	좋은 예 + 나쁜 예 각 1개	품질 향상 → 재작업 감소
이유 1줄	"왜"를 15~30단어로	예외 케이스의 자율 판단
테이블 형식	조건 → 판단 표로 작성	파싱 효율 + 판단 정확도 향상
훅 분리	강제 규칙은 훅(Hook)으로 이동	이중 관리 제거

다음 장에서는 컨텍스트 윈도우 관리 — `/clear`, `/compact`, `--resume` 의 올바른 구분 사용법을 설명한다.

제3장 컨텍스트 관리 — `/clear`, `/compact`, `--resume` 올바른 사용법

컨텍스트 윈도우는 "보이지 않는 지갑"이다. 대화가 진행될수록 잔액이 줄어들고, 바닥이면 세션이 끝난다.

이 장에서는 컨텍스트를 효율적으로 활용하기 위한 3가지 커맨드와 흔히 빠지는 함정을 설명한다.

컨텍스트 윈도우의 기본

Claude Code(Opus 4.6)의 컨텍스트 윈도우는 **1M 토큰**이다. 크게 보이지만 실제로는:

- 세션 시작 시점에 시스템 프롬프트, CLAUDE.md, 톨 정의로 수만 토큰을 소모한다
- 대화의 각 턴마다 과거 대화 기록 전체가 재전송된다
- 파일 읽기 결과, 커맨드 출력이 계속 쌓인다

체감상 일반적으로 작업하면 2~3시간 내에 75%에 도달한다. 여기서부터가 본격적인 관리 구간이다.

`/clear` — 대화 완전 초기화

`/clear` 는 대화 기록을 전부 삭제한다. 컨텍스트 윈도우가 0으로 돌아간다.

사용 시점

- 완전히 다른 작업으로 전환할 때
- 이전 대화의 맥락이 방해가 될 때 (디버깅 중 다른 버그를 추적하기 시작한 경우 등)

주의사항

`/clear` 는 **되돌릴 수 없다**. 삭제한 대화는 복구되지 않는다.

흔한 사고: 슬래시 커맨드 입력 실수. `/c` 를 입력하고 Enter를 누르면 `/clear` 와 `/compact` 중 어느 쪽이 실행될지 알 수 없다. 실제로 `/clear` 가 우선 실행되어 30분 작업이 사라진 경험이 있다.

이를 방지하기 위해 `/clear` 입력 시 블로킹하는 훅을 설정해두었다:

```
{
  "hooks": {
    "UserPromptSubmit": [{
      "matcher": "^/clear$",
      "hooks": [{"type": "command", "command": "bash /path/to/clear-command-confirm-guard.sh"}]
    }]
  }
}
```

`/clear`는 슬래시 커맨드이므로 `PreToolUse`가 아닌 `UserPromptSubmit`으로 인터셉트하는 점에 주의한다. `cc-safe-setup`의 `clear-command-confirm-guard`는 이 문제를 해결한다. `/clear`를 완전 차단하고, `/compact`로 유도하는 메시지를 표시한다.

/clear 남용 주의

`/clear`를 실행할 때마다 CLAUDE.md 재읽기, 톨 정의 재전송이 발생한다. 1세션에서 5번 `/clear`를 실행할 것 같다면, 새 세션을 시작하는 편이 낫다.

/compact — 대화를 요약하여 압축

`/compact`는 대화 기록을 요약·압축한다. 정보를 유지하면서 토큰을 확보한다.

/clear와의 비교

항목	<code>/clear</code>	<code>/compact</code>
대화 기록	완전 삭제	요약하여 보존
맥락 유지	없음	있음 (요약 정확도에 의존)
토큰 확보량	최대 (전체 해제)	중간 (요약분은 남음)
위험	정보 완전 소실	요약 시 정보 누락

사용 시점

- 컨텍스트가 75%를 넘었을 때
- "슬슬 무거워지는 것 같다"고 느낄 때
- 큰 파일을 읽은 후 (불필요해진 내용을 압축하기 위해)

수동 /compact가 auto-compact보다 효율적인 이유

Claude Code에는 `auto-compact` 기능이 있다. 컨텍스트가 약 90%에 도달하면 자동으로 압축이 실행된다.

그러나 `auto-compact`는 최악의 타이밍에 작동한다.

- 약 90%에 도달한 후 압축하므로, 압축 직전 수 톨이 가장 토큰을 많이 소모하는 구간이 된다
- 압축 처리 자체에도 토큰을 소모한다
- 압축 후 Claude Code가 "무엇을 하고 있었는지" 떠올리기 위해 추가 톨 호출을 실행하는 경우가 있다

실제 운영에서는 **75%에서 수동 /compact**를 원칙으로 삼고 있다. 일찍 압축할수록:

- 압축 전 "무거운 구간"이 짧아진다
- 요약 품질이 높아진다 (정보량이 적을 때 요약하는 편이 정확도가 높다)
- 예측 가능한 타이밍에 작업을 구분할 수 있다

/compact 전에 할 것

압축하면 세부 맥락이 빠지는 경우가 있다. 압축 전에 다음을 확인한다:

- **작업 중인 파일명을 메모한다:** 압축 후 "어떤 파일을 편집하고 있었는지" 잊는 경우가 있다
- **다음 할 작업을 명시적으로 전달한다:** `/compact` 후 `OO` 구현을 계속하세요
- **중요한 판단·결정을 기록해둔다:** `mission.md` 나 작업 파일에 미리 작성해 놓는다

`cc-safe-setup`의 `pre-compact-checkpoint` 혹은 압축 전에 자동으로 `git commit`을 생성한다. 압축 후 `git log`로 "무엇을 변경하고 있었는지" 확인할 수 있다.

--resume — 이전 세션 재개

`claude --resume`은 과거 세션을 선택하여 재개한다. `claude --continue`는 현재 디렉토리의 직전 세션을 이어간다. 둘 다 새 세션을 시작하지 않고 이전 작업의 이어서 시작할 수 있다.

사용 시점

- 중단한 작업을 이어서 할 때
- 이전 세션에서 얻은 맥락이 필요할 때
- 아침에 전날 작업을 재개할 때

주의사항

`--resume`으로 재개하면 이전 세션의 컨텍스트가 그대로 복원된다. 즉:

- 이전 세션에서 75%를 사용하고 있었다면, 재개 직후에도 75%에서 시작
- 이전 대화 전체가 컨텍스트에 포함되므로, 첫 번째 턴부터 무겁다

`--resume` 후 `/compact`를 실행하는 것이 효율적인 사용법이다:

```
claude --resume
# 재개 직후
# > /compact 어제 OO의 이어서 진행합니다
```

이렇게 하면 이전 세션의 맥락을 요약으로 보존하면서, 가벼운 컨텍스트로 작업을 시작할 수 있다.

--resume을 사용하지 말아야 할 때

- 이전과 전혀 관계없는 작업을 할 때 (새 세션으로 시작)
- 이전 세션이 예러로 어중간하게 종료되었을 때 (손상된 컨텍스트를 인계받을 위험)

흔히 빠지는 4가지 함정

함정 1: "아깝다"는 생각에 /compact를 미루기

"아직 25% 남아 있어"라며 압축을 미루는 패턴. 컨텍스트가 90%를 초과하면:

- 각 턴의 처리가 느려진다

- auto-compact가 갑작스럽게 발동해 맥락이 손실된다
- 최악의 경우 세션이 강제 종료된다

75% 룰을 지켜라.

합정 2: 큰 파일을 읽은 채 방치하기

Read로 3,000줄짜리 파일을 읽으면 그 내용이 컨텍스트에 통째로 들어간다. 작업이 진행되어 파일 내용이 불필요해져도, 컨텍스트에서 사라지지 않는다.

대처법:

- 큰 파일은 `limit` 파라미터로 필요한 부분만 읽는다
- 읽은 후, 다른 작업으로 넘어가기 전에 `/compact` 를 실행한다
- `large-read-guard` 혹은 100KB를 초과하는 파일 읽기를 사전 경고한다

합정 3: /compact 직후 대량의 정보를 한꺼번에 전달하기

압축 직후는 "아무것도 기억하지 못하는" 상태에 가깝다. 거기에 "이 파일과 저 파일을 읽고, 이 수정을 하고, 테스트를 실행하고..." 라고 한꺼번에 지시하면, 압축의 의미가 사라진다.

압축 후에는:

1. 우선 지시 하나를 전달한다
2. 결과를 확인한다
3. 다음 지시를 전달한다

서두르지 말고 1단계씩 진행하는 편이 토큰 효율이 좋다.

합정 4: auto-compact 후 폭주

auto-compact 후, Claude Code가 압축 전 대화를 "요약"으로 읽고 임의로 행동을 시작하는 경우가 있다. "이전 대화에서 ○○를 하겠다고 했으니, 시작하자"는 패턴이다.

이는 의도하지 않은 변경을 유발하는 원인이 된다. `cc-safe-setup` 의 `pre-compact-checkpoint` 혹은 압축 전에 체크포인트를 저장함과 동시에, "압축 후에는 지시를 확인하고 나서 행동하세요"라는 메시지를 주입한다. 이로써 압축 후 폭주를 억제할 수 있다.

실전: 세션 설계 템플릿

효율적인 세션의 흐름:

1. 세션 시작 (또는 `--resume`)
2. 작업을 명확하게 전달
3. 작업 (컨텍스트 0% → 75%)
4. 수동 `/compact` "다음은 OO를 진행합니다"
5. 작업 (컨텍스트 30% → 75%)
6. 수동 `/compact`
7. 작업
8. 필요에 따라 `/compact` 1회 더
9. 세션 종료

1세션에서 2~3회 `/compact`가 표준적인 페이스다.

auto-compact에 3회 이상 의존하고 있다면, 수동 `/compact` 타이밍이 너무 늦은 것이다.

.claudeignore로 불필요한 파일 제외하기

`.claudeignore`는 `.gitignore`와 동일한 문법으로, Claude Code가 읽지 않을 파일을 지정한다. 프로젝트 루트에 두기만 하면 바로 적용된다.

```
# .claudeignore 예시
node_modules/
dist/
build/
*.log
*.lock
coverage/
.next/
```

이것은 "레버리지가 가장 높은 설정 중 하나"로 꼽힌다. 이유는 명확하다: Claude Code는 지시 없이도 프로젝트 구조를 파악하기 위해 파일을 읽는다. 빌드 산출물이나 `node_modules`를 읽으면, 1회의 Read로 수천~수만 토큰을 소모한다. `.claudeignore`로 제외하면 이 소모가 0이 된다.

`.gitignore`와 `.claudeignore`가 모두 있는 경우, 양쪽 규칙이 모두 적용된다. 이미 `.gitignore`로 빌드 산출물을 제외하고 있다면, `.claudeignore`에는 Claude 고유 제외 규칙(대용량 문서 폴더, 테스트 픽스처 등)을 추가하는 것이 효율적이다.

MCP 서버의 컨텍스트 비용

MCP 서버를 연결하면 해당 툴 정의(스키마)가 컨텍스트에 주입된다. 이는 요청과 응답 양쪽에서 발생한다.

예를 들어, GitHub MCP 서버를 연결하면 30개 이상의 툴 정의가 매 턴 전송된다. 사용하지 않는 툴의 스키마만으로 수천 토큰을 소모한다. 동일한 작업을 `gh` CLI(Bash 경유)로 처리할 수 있다면, CLI 쪽이 토큰 효율이 훨씬 좋다.

대처법:

- 사용하지 않는 MCP 서버는 `settings.json`에서 제거한다
- CLI로 동일하게 처리할 수 있다면 MCP보다 CLI를 우선한다

- MCP 서버는 "**CLI**로는 불가능한 작업"에만 한정한다

다음 장에서는 훅(Hook)을 사용해 토큰 낭비 패턴을 자동으로 감지·차단하는 방법을 설명한다.

제4장 훅(Hook)으로 토큰 낭비를 자동 차단하라

CLAUDE.md에 "불필요한 파일을 읽지 마라"고 써도 Claude Code는 읽는 경우가 있다. 훅(Hook)은 읽지 않는다.

훅은 Claude Code의 각 작업 전후에 자동 실행되는 셸 스크립트다. 규칙을 "부탁"에서 "강제"로 바꾼다. 이 장에서는 토큰 절약에 직결되는 9개의 훅을 소개한다.

훅(Hook)의 기본

훅은 `settings.json`에 설정한다. 위치는 `~/.claude/settings.json` (글로벌) 또는 프로젝트의 `.claude/settings.json`.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{"type": "command", "command": "/path/to/your-hook.sh"}]
      }
    ]
  }
}
```

- **PreToolUse:** 툴 실행 전에 작동한다. `exit 2`를 반환하면 툴 실행을 차단한다
- **PostToolUse:** 툴 실행 후에 작동한다. 결과 모니터링이나 로그 기록에 사용한다
- **PreCompact:** `/compact` 전에 작동한다. 맥락 보존에 사용한다
- **SessionStart:** 세션 시작 시 작동한다. 초기 상태 설정에 사용한다

훅의 stdout에 출력한 내용은 Claude Code의 컨텍스트에 주입된다. 즉, "이 파일은 너무 큼니다. limit 파라미터를 사용해 주세요"라는 메시지를 훅에서 출력하면 Claude Code가 그에 따른다.

⚠ v2.1.101 이후 신기능 (구현 시기는 변경될 수 있음): PreToolUse 훅에 `defer` 판정이 추가되었다. 훅의 stdout에 JSON `{"decision": "defer"}`를 반환하면 헤드리스 세션이 일시 정지되고, 재개 시 재평가된다. 자율 운영 중 "이 작업은 사람이 확인해야 한다"는 상황에서 토큰 낭비를 방지할 수 있다. 사용 중인 버전에서 이용 가능한지 확인할 것.

본 장의 훅에서는 `$PPID` (부모 프로세스 ID)를 세션 식별자로 사용한다. Claude Code가 훅을 실행할 때, 훅의 bash 프로세스 관점에서 본 `$PPID`는 Claude Code의 프로세스 ID가 되므로, 동일 세션 내 훅 호출을 추적할 수 있다.

훅 1: 대용량 파일 읽기 가드

가장 토큰을 많이 소모하는 것은 큰 파일을 통째로 읽는 것이다.

```
#!/bin/bash
# large-read-guard.sh - 100KB 초과 파일 읽기 경고

# 혹은 stdin은 JSON 형식. tool_input 내에 톨의 인수가 있다
input=$(cat)
file_path=$(echo "$input" | jq -r '.tool_input.file_path // empty')

if [ -n "$file_path" ] && [ -f "$file_path" ]; then
    size=$(stat -c%s "$file_path" 2>/dev/null || stat -f%z "$file_path" 2>/dev/null)
    threshold=${CC_LARGE_READ_KB:-100}
    threshold_bytes=$((threshold * 1024))

    if [ "$size" -gt "$threshold_bytes" ] 2>/dev/null; then
        size_kb=$((size / 1024))
        echo "⚠️ ${file_path} 는 ${size_kb}KB 입니다."
        echo "limit 파라미터로 필요한 부분만 읽어 주세요."
        echo "예: Read file_path=${file_path} limit=100"
        exit 0 # 경고만 (stdout 내용이 Claude Code에 전달된다)
    fi
fi
```

설정:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Read",
      "hooks": [{"type": "command", "command": "bash /path/to/large-read-guard.sh"}]
    }]
  }
}
```

효과: 3,000줄짜리 파일을 통째로 읽는 대신 필요한 100줄만 읽게 된다. 1회 Read로 수천 토큰 절약.

 이후의 훅도 동일한 형식으로 `settings.json`에 설정한다. 이벤트명과 matcher는 이 장 말미의 요약표를 참고한다. 훅 8·9에는 설정 예시를 별도로 기재했다.

훅 2: 읽기 파일 수 상한선

1세션에서 100개 이상의 파일을 읽는 것은 거의 확실히 비효율이다.

```
#!/bin/bash
# read-budget-guard.sh - 세션 중 읽기 파일 수 제한

budget=${CC_READ_BUDGET:-100}
warn=${CC_READ_WARN:-50}
tracker="/tmp/cc-read-budget-$$PID"
```

```

input=$(cat)
file_path=$(echo "$input" | jq -r '.tool_input.file_path // empty')

# 카운트
count=0
if [ -f "$tracker" ]; then
    if ! grep -qF "$file_path" "$tracker"; then
        echo "$file_path" >> "$tracker"
    fi
    count=$(wc -l < "$tracker")
else
    echo "$file_path" > "$tracker"
    count=1
fi

if [ "$count" -ge "$budget" ]; then
    echo "🔴 읽기 파일 수가 ${budget}개에 도달했습니다."
    echo "Glob/Grep으로 범위를 좁힌 후 읽어 주세요."
    exit 2 # exit 2 = 차단
elif [ "$count" -ge "$warn" ]; then
    echo "⚠️ ${count}/${budget} 파일 읽기 완료. 남은 횟수: $((budget - count))개."
fi

```

효과: "혹시 몰라서 전부 읽는" 패턴 방지. Grep이나 Glob으로 필요한 파일을 특정한 후 읽는 습관이 형성된다.

흑 3: 토큰 예산 가드

세션 전체의 토큰 소비를 추정하고, 예산을 초과하면 경고한다.

```

#!/bin/bash
# token-budget-guard.sh - 세션의 토큰 예산 관리

budget_warn=${CC_TOKEN_BUDGET:-50000}
budget_block=${CC_TOKEN_BLOCK:-100000}
tracker="/tmp/cc-token-budget-$$PID"

# 톨 출력 크기로 토큰 수를 추정
# (영어 텍스트 기준 약 4자=1토큰. 한국어나 코드에서는 비율이 달라 어디까지나 목안)
input=$(cat)
output_size=${#input}
tokens=$((output_size / 4))

total=0
if [ -f "$tracker" ]; then
    total=$(cat "$tracker")
fi
total=$((total + tokens))
echo "$total" > "$tracker"

if [ "$total" -ge "$budget_block" ]; then

```

```

echo "🔴 추정 토큰 소비가 ${budget_block}을 초과했습니다. /compact를 실행해 주세요."
exit 2 # exit 2 = 차단
elif [ "$total" -ge "$budget_warn" ]; then
    echo "⚠️ 추정 토큰 소비: ${total}/${budget_block}"
fi

```

효과: "어느새 컨텍스트가 90%" 상황 방지. 일찍 `/compact` 를 실행하도록 유도한다.

흑 4: 톨 호출 빈도 제한

30초 안에 15회 이상의 톨 호출은 폭주의 신호다.

```

#!/bin/bash
# token-spike-alert.sh - 톨 호출의 이상 빈도 감지

tracker="/tmp/cc-spike-$$"
window=30 # 초
threshold=15

now=$(date +%s)
echo "$now" >> "$tracker"

# window초 이내 항목만 남긴다
cutoff=$((now - window))
if [ -f "$tracker" ]; then
    awk -v cutoff="$cutoff" ' $1 >= cutoff ' "$tracker" > "${tracker}.tmp"
    mv "${tracker}.tmp" "$tracker"
    count=$(wc -l < "$tracker")

    if [ "$count" -ge "$threshold" ]; then
        echo "⚠️ ${window}초 동안 ${count}회 톨 호출 감지. 폭주 가능성이 있습니다."
        echo "의도한 것이 아니라면 Ctrl+C로 중단해 주세요."
    fi
fi

```

효과: 루프 상태(같은 파일을 반복해서 읽기, 같은 커맨드를 반복 실행)의 조기 감지.

흑 5: 서브에이전트 수 제한

서브에이전트는 편리하지만, 하나 실행할 때마다 독립적인 컨텍스트가 생성된다.

```

#!/bin/bash
# subagent-budget-guard.sh - 동시 서브에이전트 수 제한

max=${CC_MAX_SUBAGENTS:-5}
tracker="/tmp/cc-subagent-$$"

now=$(date +%s)

```

```

echo "$now" >> "$tracker"

# 최근 5분 이내에 실행된 에이전트 수 카운트
cutoff=$((now - 300))
if [ -f "$tracker" ]; then
    count=$(awk -v cutoff="$cutoff" ' $1 >= cutoff ' "$tracker" | wc -l)

    if [ "$count" -gt "$max" ]; then
        echo "🔴 5분 동안 서버에이전트 ${count}개가 실행되었습니다 (상한: ${max}개)."
        echo "메인 컨텍스트에서 Glob/Grep을 사용해 주세요."
        exit 2 # exit 2 = 차단
    fi
fi

```

효과: 서버에이전트 남용 방지. 단순 검색에 서버에이전트를 사용하는 패턴을 억제한다.

혹 6: Bash 출력 크기 제한

`ls -la` 출력은 수십 줄이지만, `git log` 는 수천 줄이 될 수 있다.

```

#!/bin/bash
# bash-output-guard.sh - Bash 출력이 너무 큰 경우 경고

input=$(cat)
command=$(echo "$input" | jq -r '.tool_input.command // empty')

# 출력이 커질 수 있는 커맨드 감지
dangerous_patterns="git log|find / |cat |strings |xxd |hexdump"
if echo "$command" | grep -qE "$dangerous_patterns"; then
    if ! echo "$command" | grep -qE "\| head|\| tail|\| grep|--max-count|-n [0-9]"; then
        echo "⚠ 이 커맨드의 출력이 커질 수 있습니다."
        echo "| head -50 또는 --max-count 옵션을 추가해 주세요."
    fi
fi

```

효과: 무제한 커맨드 출력으로 인한 컨텍스트 압박 방지.

혹 7: 중복 파일 읽기 감지

같은 파일을 두 번 읽는 것은 거의 항상 낭비다.

```

#!/bin/bash
# duplicate-read-detector.sh - 동일 파일 재읽기 경고

tracker="/tmp/cc-read-history-$$PID"
input=$(cat)
file_path=$(echo "$input" | jq -r '.tool_input.file_path // empty')

```

```

if [ -n "$file_path" ] && [ -f "$tracker" ]; then
    if grep -qF "$file_path" "$tracker"; then
        echo "⚠️ ${file_path} 는 이미 읽은 파일입니다."
        echo "내용은 컨텍스트에 있습니다. 재읽기가 불필요할 수도 있습니다."
        # 차단하지 않음 (파일이 업데이트된 경우도 있으므로)
    fi
fi

echo "$file_path" >> "$tracker"

```

효과: "방금 읽은 파일을 다시 읽는" 패턴을 가시화한다.

훅 8: /compact 전 자동 체크포인트

/compact 전에 작업 상태를 자동으로 저장한다.

```

#!/bin/bash
# pre-compact-checkpoint.sh - /compact 전 git commit

cd "$(git rev-parse --show-toplevel 2>/dev/null)" || exit 0

if [ -n "$(git status --porcelain)" ]; then
    git add -A
    # --no-verify: 체크포인트 전용이므로 pre-commit 훅을 건너뛸
    git commit -m "checkpoint: pre-compact $(date +%Y%m%d-%H%M%S)" --no-verify 2>/dev/null
    echo "✅ 압축 전 체크포인트를 저장했습니다."
fi

echo "📁 컨텍스트가 압축됩니다."
echo "압축 후에는 이전 대화의 맥락이 사라집니다. 압축 전 작업을 자동으로 재개하지 말고, 사용자의 지시를 먼저 확인해 주세요."

```

설정:

```

{
  "hooks": {
    "PreCompact": [{
      "matcher": "",
      "hooks": [{"type": "command", "command": "bash /path/to/pre-compact-checkpoint.sh"}]
    }]
  }
}

```

효과: 압축 후 `git log` 로 "무엇을 변경하고 있었는지" 확인 가능.

훅 9: 세션 시작 시 예산 표시

세션 시작 시 오늘의 남은 토큰 예산을 표시한다.

```
#!/bin/bash
# session-budget-alert.sh — 세션 시작 시 예산 현황 표시

state_file="/tmp/cc-daily-budget"
today=$(date +%Y-%m-%d)

if [ -f "$state_file" ]; then
    saved_date=$(head -1 "$state_file")
    if [ "$saved_date" = "$today" ]; then
        sessions=$(tail -1 "$state_file")
        echo "📊 오늘 ${sessions}번째 세션입니다. 토큰 소비에 주의해 주세요."
        echo "$today" > "$state_file"
        echo "$((sessions + 1))" >> "$state_file"
        exit 0
    fi
fi

echo "$today" > "$state_file"
echo "1" >> "$state_file"
echo "📊 오늘 첫 번째 세션입니다. 좋은 시작을."
```

설정:

```
{
  "hooks": {
    "SessionStart": [{
      "matcher": "",
      "hooks": [{"type": "command", "command": "bash /path/to/session-budget-alert.sh"}]
    }]
  }
}
```

정리: 9개 훅 요약표

훅	이벤트	효과
대용량 파일 가드	PreToolUse (Read)	100KB 초과 통째 읽기 경고
읽기 예산	PreToolUse (Read)	세션당 100개 파일 상한
토큰 예산	PostToolUse	추정 소비량으로 /compact 유도
스파이크 감지	PostToolUse	30초 15회로 폭주 경고
서브에이전트 제한	PreToolUse (Agent)	5분 5개까지
Bash 출력 제한	PreToolUse (Bash)	대출력 커맨드에 head 권장
중복 읽기 감지	PreToolUse (Read)	동일 파일 재읽기 경고
compact 전 체크포인트	PreCompact	자동 git commit + 폭주 방지 메시지
세션 예산 표시	SessionStart	일별 세션 수 표시

이 훅들을 전부 직접 작성할 필요는 없다. `cc-safe-setup`에는 667개 이상의 예시 훅이 준비되어 있으며, `--install-example` 커맨드로 개별 설치가 가능하다.

⚠ 주의: 본 장의 훅은 `cc-safe-setup`의 example과는 독립된 구현입니다. `--install-example`로는 설치할 수 없습니다. 본 장의 훅을 사용하는 경우 `settings.json`에 수동으로 추가해 주세요.

같은 이름의 훅이라도 설계가 다릅니다.

- **large-read-guard:** 본 장은 Read 톨 대상(`matcher: "Read"`), `cc-safe-setup`은 Bash(cat) 대상
- **token-budget-guard:** 본 장은 토큰 수 기준 관리, `cc-safe-setup`은 달러 환산 기준

두 가지를 동시에 사용하지 마세요. 반드시 하나만 선택하세요.

다음 장에서는 서브에이전트를 전략적으로 활용하는 방법을 설명한다.

제5장 서브에이전트 전략 — 메인 컨텍스트를 지켜라

서브에이전트는 Claude Code 내부에서 별도의 Claude Code를 실행하는 기능이다. `Agent` 톨로 호출되며, 독자적인 컨텍스트 윈도우에서 동작한다.

메인 컨텍스트를 오염시키지 않고 병렬 처리할 수 있다. 단, 잘못 사용하면 톨 소비가 폭발적으로 늘어난다.

서브에이전트의 동작 방식

메인 Claude Code가 `Agent` 톨을 호출하면:

1. 새로운 Claude Code 프로세스가 실행된다
2. 프롬프트(지시)가 해당 프로세스에 전달된다
3. 서브에이전트가 독자적으로 작업한다 (파일 읽기, 검색, 커맨드 실행)
4. 결과가 메인으로 반환된다

핵심은 **서브에이전트의 작업 내용이 메인 컨텍스트에 쌓이지 않는다**는 점이다. 메인에는 "서브에이전트를 실행했다"와 "결과는 이랬다"만 기록된다.

즉, 서브에이전트가 50개 파일을 읽어도, 메인 컨텍스트는 50개 파일 분량으로 비대해지지 않는다. 결과 요약만 들어온다.

언제 서브에이전트를 사용해야 하는가

✅ 사용해야 할 상황

- 광범위한 검색: "이 에러 메시지가 어느 파일에서 발생하는지 찾아줘"
- 병렬 조사: "A와 B와 C, 세 가지 접근법을 각각 조사해서 비교해줘"
- 대량 파일 처리: "전체 테스트 파일에서 더 이상 사용되지 않는 API 사용 위치를 찾아줘"
- 독립적인 작업: 메인 작업과 관계없는 확인 작업

❌ 사용하지 말아야 할 상황

- 1개 파일 검색: Grep이나 Glob으로 직접 하는 것이 빠르고 저렴하다
- 단순한 커맨드 실행: Bash로 직접 실행해야 한다
- 메인의 맥락이 필요한 작업: 서브에이전트는 메인의 대화 기록을 알지 못한다

서브에이전트의 톨 큰 비용

서브에이전트의 비용을 추정하면:

서브에이전트 비용 = 시스템 프롬프트 + 프롬프트(지시) + 작업 중 톨 호출 + 결과 생성

최소 수천 토큰은 소모된다. "이 파일의 10번째 줄을 읽어줘" 수준의 작업에 서브에이전트를 사용하면, 직접 Read하는 것보다 10배 이상의 토큰을 소비한다.

비용 비교표

작업	직접 실행	서브에이전트 경유
파일 1개 검색	~100 토큰	~3,000 토큰
파일 10개 조사	~1,000 토큰 (컨텍스트에 남음)	~5,000 토큰 (컨텍스트에 남지 않음)
파일 50개 조사	~5,000 토큰 (컨텍스트 비대화)	~8,000 토큰 (컨텍스트는 깨끗하게 유지)

※ 위 수치는 1200시간 운영 중 관찰한 대략적인 값이다. 실제 토큰 소비는 파일 크기, 프롬프트 길이, 모델 응답량에 따라 변동한다.

분기점은 10~20개 파일이다. 그 이하라면 직접 처리하는 편이 저렴하다. 그 이상이라면 서브에이전트가 메인 컨텍스트를 지킬 수 있어 유리해진다.

5가지 전략

전략 1: 서브에이전트 1개 = 작업 1개

서브에이전트에 여러 작업을 한꺼번에 맡기지 않는다.

✗ 나쁜 예:

"파일 구조를 조사하고, 테스트를 실행하고, 문서도 업데이트해줘"

✓ 좋은 예:

서브에이전트 1: "src/ 이하의 파일 구조를 조사해서 리포트해줘"

서브에이전트 2: "테스트를 실행하고 실패 위치를 알려줘"

(문서 업데이트는 메인에서 실행)

작업 하나에 집중시키면:

- 프롬프트가 명확해져 불필요한 시행착오가 줄어든다
- 하나가 실패해도 다른 것에 영향을 주지 않는다
- 결과가 간결해져 메인 컨텍스트를 압박하지 않는다

전략 2: 프롬프트에 충분한 맥락을 전달한다

서브에이전트는 메인의 대화 기록을 알지 못한다. 필요한 정보는 프롬프트에 포함해야 한다.

✗ 나쁜 예:

"아까 이야기한 버그를 수정해줘"
→ 서브에이전트는 "아까의 대화"를 모른다

✅ 좋은 예:

"src/api/handler.ts 의 getUserById 함수에서
id 가 undefined 일 때 TypeError 가 발생한다.
null 체크를 추가해서 수정해줘"
→ 서브에이전트가 즉시 작업할 수 있다

단, 프롬프트가 너무 길어도 문제다. **100자 미만은 너무 모호하고, 1,000자 이상은 지나치게 장황하다. 200~500자가 기준이다.**

`cc-safe-setup`의 `subagent-context-size-guard` 혹은 100자 미만 프롬프트를 경고한다.

전략 3: 병렬 실행을 적극 활용한다

독립적인 작업이라면 서브에이전트를 여러 개 동시에 실행할 수 있다.

메인: "A와 B를 동시에 조사해줘"
→ 서브에이전트 A: A 조사
→ 서브에이전트 B: B 조사
(병렬 실행)
메인: 양쪽 결과를 받아서 판단

직렬 방식이라면 "A 조사 → 결과 확인 → B 조사 → 결과 확인"으로 4단계가 필요하지만, 병렬이면 2단계로 끝난다. 토큰 소비는 거의 같지만 **작업 시간이 절반**이 된다.

전략 4: 탐색은 서브에이전트, 구현은 메인

서브에이전트도 파일 편집이 가능하지만, 메인의 맥락 없이 편집하면 의도하지 않은 변경이 생기기 쉽다.

권장 패턴:

서브에이전트: "이 문제의 원인을 조사해서 리포트해줘"
→ 조사 결과를 반환

메인: 조사 결과를 읽고, 직접 수정을 구현한다

조사로 인한 컨텍스트 비대화를 방지하면서, 구현은 메인의 전체적인 이해를 바탕으로 진행한다.

전략 5: 동시 실행 수를 제한한다

5분 안에 5개 이상의 서브에이전트는 너무 많다.

- 서브에이전트 1개 = 수천 토큰
- 5개 동시 = 1만 토큰 이상이 한순간에 소비
- 10개 동시 = 쿼터에 직접적인 타격

`subagent-budget-guard` 혹은로 상한선을 설정해두는 것이 안전하다.

흔히 빠지는 4가지 함정

함정 1: 단순 검색에 서브에이전트를 사용한다

- ❌ "'handleError'가 사용된 위치를 찾아줘"
→ 서브에이전트 실행 → 내부에서 Grep 실행 → 결과 반환
- ✅ 직접 Grep을 실행
→ 결과가 즉시 나온다. 비용은 1/30

Grep, Glob, Read(limit 포함)로 처리할 수 있는 작업에 서브에이전트를 사용하지 않는다.

함정 2: 서브에이전트의 결과를 너무 신뢰한다

서브에이전트는 메인의 맥락을 알지 못한다. "문제없습니다"라는 보고가 메인의 맥락에서는 틀린 경우가 있다.

서브에이전트의 결과는 "참고 정보"로 받아들이고, 최종 판단은 메인에서 내린다.

함정 3: 서브에이전트에 CLAUDE.md 내용이 전달되지 않는다

Claude Code의 버전에 따라서는 서브에이전트에 CLAUDE.md 내용이 자동으로 전달되지 않는 경우가 있다.

중요한 규칙(코딩 규약, 금지 작업 등)은 서브에이전트의 프롬프트에도 명시하거나, `subagent-claudemd-inject` 혹은로 CLAUDE.md 내용을 자동 주입한다.

함정 4: 병렬 서브에이전트의 파일 충돌

2개 이상의 서브에이전트가 같은 파일을 건드리면 "File modified since read" 에러가 발생한다.

실제 사례 (GitHub Issue #46968):

병렬 서브에이전트가 같은 파일을 읽고 쓰려 했다
"File modified since read" 에러가 연속 발생
각 서브에이전트가 재시도를 반복해 101,000 토큰 이상이 재시도 루프에 소비
추가로 세션 전환으로 150,000 토큰 이상이 완전히 손실

대처법:

- 병렬 서브에이전트에는 겹치지 않는 파일 범위를 지정한다
- "src/api/를 조사해줘", "src/models/를 조사해줘" 처럼 작업 범위를 명확히 분리한다
- 같은 파일에 쓰기가 필요하다면 직렬(순서대로 1개씩)로 처리한다

실전: 서브에이전트 사용 판단 플로우

작업을 의뢰하고 싶다

- └ 1~2개 파일의 검색·읽기 → 직접 실행
- └ 3~10개 파일의 조사 → 직접 실행 (컨텍스트 여유가 있으면)
- └ 10개 이상의 광범위한 조사 → 서브에이전트 사용
- └ 독립적인 병렬 작업 → 서브에이전트 (여러 개 동시)
- └ 메인의 맥락이 필요한 작업 → 메인에서 실행
- └ 편집·구현 → 메인에서 실행 (조사만 서브에이전트에게)

이 판단을 매번 의식하는 것만으로도 서브에이전트 남용이 크게 줄어든다.

다음 장에서는 작업 분할과 실행 순서로 토큰 효율을 높이는 워크플로우 설계를 설명한다.

제6장 워크플로우 설계 — 작업 분할과 실행 순서로 토큰 효율을 높인다

같은 결과를 내더라도 방법에 따라 토큰 소비가 2배 달라진다. 이 장에서는 작업 분할 방법과 실행 순서로 토큰 효율을 개선하는 방법을 설명한다.

원칙: 1세션 = 1테마

세션 도중에 테마를 바꾸면 이전 테마의 맥락이 낭비된다.

❌ 비효율적인 세션:

1. 버그 수정 (파일 A, B, C를 읽는다)
 2. 문서 업데이트 (파일 D, E를 읽는다)
 3. 테스트 추가 (파일 A, B, F를 읽는다)
- 3개 테마의 컨텍스트가 혼재. 버그 수정에서 읽은 정보는 문서 업데이트에 불필요

✅ 효율적인 세션 설계:

- 세션 1: 버그 수정 (A, B, C → 수정 → 테스트)
세션 2: 문서 업데이트 (D, E → 업데이트)
세션 3: 테스트 추가 (A, B, F → 테스트 작성)
→ 각 세션이 1테마에 집중. 컨텍스트 낭비가 적다

"그래도 세션을 나누면 시작 비용이 드는 것 아닌가?" — 든다. 그러나 혼재된 컨텍스트로 75%에 도달해 `/compact` 를 실행하는 편이 총합으로는 더 싸다.

작업 분할의 3가지 패턴

패턴 1: 파일군으로 나누기

관련 파일을 묶어서 1세션에서 처리한다.

프로젝트 구성:

```
src/  
  api/      ← 세션 1: API 레이어 수정  
  db/       ← 세션 2: DB 레이어 수정  
  ui/       ← 세션 3: UI 레이어 수정  
tests/     ← 각 세션 마지막에 해당 테스트 실행
```

한 레이어를 수정할 때, 해당 레이어의 파일만 읽으면 된다. 다른 레이어의 파일은 컨텍스트에 넣지 않는다.

패턴 2: 공정으로 나누기

조사 → 설계 → 구현 → 테스트 → 리뷰를 별도 세션으로 진행한다.

세션 1 (조사):

- 관련 파일을 읽는다
- 문제의 원인을 파악한다
- 수정 방침을 결정하고 메모에 작성한다 (mission.md 또는 작업 파일)

세션 2 (구현):

- 메모에서 수정 방침을 읽는다
- 구현한다
- 테스트를 실행한다

세션 3 (리뷰):

- git diff로 변경 사항을 확인한다
- 품질 체크

공정을 나누면 각 세션에서 필요한 컨텍스트가 최소화된다. 조사 세션에서 읽은 50개 파일은 구현 세션에서 불필요하다. 수정 방침 메모만 있으면 충분하다.

패턴 3: 규모로 나누기

큰 작업을 작은 작업으로 분할한다.

❌ 1세션에서:

"인증 시스템을 구현해줘"

→ 인증 로직 + DB + API + UI + 테스트 → 컨텍스트 초과

✅ 4세션으로 나눠서:

1. "인증 로직의 핵심 부분을 구현해줘"
2. "DB 레이어와 마이그레이션을 만들어줘"
3. "API 엔드포인트를 만들어줘"
4. "UI 로그인 폼을 만들어줘"

각 세션이 독립적이므로 도중에 중단해도 재개하기 쉽다.

실행 순서 최적화

"읽기" 먼저, "쓰기" 나중에

같은 파일군에 대해 읽기와 쓰기가 모두 필요한 경우, 먼저 전부 읽고 나서 쓰는 편이 효율적이다.

❌ 파일별 읽기→쓰기:

Read A → Edit A → Read B → Edit B → Read C → Edit C

→ A를 읽은 후 B를 읽으면, A의 내용은 컨텍스트에 남지만 사용하지 않는다

✅ 전부 읽고 나서 일괄 쓰기:

Read A → Read B → Read C → Edit A → Edit B → Edit C
→ 읽기 단계에서 전체상을 파악. 쓰기 단계는 판단이 빠르다

단, 파일 수가 많으면 컨텍스트 압박에 주의한다. 10개 이상이라면 분할을 검토한다.

의존성을 고려한다

A가 B에 의존하는 경우, B를 먼저 수정한다.

✅ 의존성에 따른 순서:

1. 저수준 (유틸리티 함수)을 수정
 2. 중간 수준 (비즈니스 로직)을 수정
 3. 고수준 (API/UI)을 수정
- 각 단계에서 이전 단계의 수정 결과를 확인하며 진행할 수 있다

역순으로 진행하면 고수준 수정 후 저수준을 변경해 고수준이 깨지는 불필요한 사이클이 발생한다.

CLAUDE.md로 워크플로우 지시하기

자주 사용하는 워크플로우를 CLAUDE.md에 작성해두면, 매번 지시하는 수고와 토큰을 절약할 수 있다.

작업 절차

1. 변경 전 `git checkout -b feature/xxx`
2. 변경 대상 파일을 Read (limit 포함)
3. 편집은 Edit으로 최소한으로
4. 테스트 실행
5. `git commit` (메시지에 "왜"를 포함)

5줄로 매 세션마다 지시가 불필요해진다. "테스트해줘", "커밋해줘"라고 매번 말할 필요가 없다.

작업 인수인계

세션을 나뉘었을 때 이전 세션의 결과를 다음 세션에 효율적으로 이어주는 방법.

방법 1: 작업 파일

```
# tasks/auth-implementation.md

## 완료
- [x] 인증 로직 코어 (src/auth/core.ts)
- [x] DB 스키마 (src/db/migrations/001_auth.sql)

## 다음에 할 것
- [ ] API 엔드포인트 (src/api/auth.ts)
```

- POST /login, POST /register, GET /me
- core.ts의 authenticate()를 사용
- JWT 토큰 유효 기간은 1시간

주의 사항

- 비밀번호는 bcrypt (cost=12)
- rate limit은 IP당 10회/분

다음 세션에서 `Read tasks/auth-implementation.md` 만 하면 맥락이 복원된다. 이전 세션의 대화를 전부 이어받는(`--resume`)것보다 훨씬 가볍다.

방법 2: git commit 메시지

```
git log --oneline -5
# abc1234 feat(auth): add JWT token generation (core.ts)
# def5678 feat(db): add auth tables migration
# ghi9012 chore: add bcrypt dependency
```

`git log` 는 수십 토큰으로 "무엇을 했는지"를 파악할 수 있다. 코드 변경 내역이 필요하면 `git diff abc1234..HEAD` 로 확인한다.

방법 3: /compact의 커스텀 요약

/compact 인증 시스템 구현 중. core.ts와 DB 완료. 다음은 API 엔드포인트.

/compact 에 메시지를 붙이면 해당 메시지가 압축 후 요약에 포함된다. 다음 턴에도 맥락이 남는다.

흔히 빠지는 3가지 함정

함정 1: "하는 김에"가 쌓인다

버그 수정 중 "하는 김에 이 코드도 리팩토링하자", "하는 김에 테스트도 추가하자". 하나하나는 작지만, 쌓이면 컨텍스트가 비대해진다.

"하는 김에"라는 생각이 들면, 작업 파일에 메모해두고 별도 세션에서 진행한다.

함정 2: 완료 확인을 하지 않는다

작업 완료 여부를 확인하지 않고 다음으로 넘어가면, 나중에 돌아와서 다시 확인하는 상황이 된다.

✅ 작업 1개마다 완료 확인:

1. 구현한다
2. 테스트 실행 → pass
3. git commit
4. 다음 작업으로

❌ 한꺼번에 확인:

1. A를 구현
2. B를 구현
3. C를 구현
4. 테스트 실행 → B가 깨짐 → B를 다시 읽는다 (토큰 낭비)

합정 3: 상태 관리 파일의 과도한 업데이트

`mission.md` 나 `todo.md` 를 툴 호출마다 업데이트하는 패턴. 실제 운영 환경에서 이 과도한 업데이트가 세션 활동의 약 30%를 차지했던 시기가 있었다.

상태 관리 파일 업데이트는 다음 3가지 타이밍으로 충분하다:

- 작업 완료 시
- 중요한 판단·방침 변경 시
- 세션 종료 전

피크 시간대를 의식한다

Max 플랜에는 피크 시간대(태평양 표준시 오전 5시~11시 / 한국 시간 기준 오후 9시~익일 오전 3시 전후)의 소비 조정이 있다. 같은 작업이라도 피크 시간대에는 비피크 시간대 대비 쿼터 소비가 체감상 35~40% 더 많다.

⚠ 추가된 내용: 원문의 피크 시간은 일본 시간 기준으로 표기되어 있으나, 한국 독자를 위해 한국 표준시 (KST) 기준으로 변환하여 병기했습니다.

이것은 설정으로 회피할 수 있는 것이 아니라, Anthropic의 서버 사이드 조정이다. 대처법은:

- 무거운 작업 (대량 파일 조작, 서브에이전트 병렬 실행)은 피크 시간대 외에 진행한다
- 피크 시간대에는 가벼운 작업(리뷰, 문서 업데이트, 계획 수립)을 우선한다
- 자율 운영의 경우 `nohup` 스케줄을 피크 시간대 외로 설정한다

다음 장에서는 1200시간 운영에서 얻은 실제 토큰 소비 데이터를 공개한다.

제7장 1200시간 소비 데이터 — 토큰을 어디에 사용했는가

버전 정보: 이 내용은 2026년 4월 기준 Claude Code(v2.1.x / Max 플랜)에 기반한 실측 데이터입니다.

이론보다 데이터. 이 장에서는 1200시간 이상의 자율 운영에서 기록한 토큰 소비 실태를 공개한다.

운영 개요

항목	내용
기간	2025년 12월 ~ 2026년 4월 (누계 약 1200시간)
플랜	Max 플랜 (월 \$200)
용도	아티클 작성, 툴 개발, 품질 관리, GitHub Issue 대응
세션 수	240개 이상
1세션 평균 시간	2~4시간

⚠ 추가된 내용: 원문의 운영 개요 항목을 한국 독자 가독성을 위해 표 형식으로 재구성했습니다.

소비가 컸던 패턴 Top 5

🥇 1위: 상태 관리 파일의 과도한 업데이트 (세션 활동의 약 30%)

가장 낭비가 심했던 것은 `mission.md` 나 `todo.md` 의 빈번한 업데이트였다.

초기 설계에서는 툴 호출마다 상태 파일을 업데이트하고 있었다. 1세션에서 100회 이상의 Read와 Write가 상태 관리에만 소모되었다.

개선: 상태 업데이트를 "작업 완료 시", "방침 변경 시", "세션 종료 전"의 3가지 타이밍으로 한정. 세션 활동에서 상태 관리가 차지하는 비율이 약 30% → 10% 미만으로 감소.

🥈 2위: 대용량 파일 통째 읽기

`package-lock.json` (수천 줄), 로그 파일, 빌드 산출물 등 본래 읽을 필요가 없는 큰 파일을 통째로 Read하고 있었다.

개선: `large-read-guard` 훅으로 100KB 초과를 경고. `limit` 파라미터 사용을 철저히 준수.

🏆 3위: 같은 파일의 반복 읽기

1세션 내에서 같은 파일을 3~4회 읽는 패턴. 특히 설정 파일(`settings.json`, `config.yaml`)에서 많이 발생.

원인: `/compact` 후 "그 파일 내용이 뭐였지?"를 떠올리기 위해 다시 읽음.

개선: 중요한 설정 값은 작업 파일에 메모해둔다. `duplicate-read-detector` 흑으로 가시화.

4위: 서브에이전트 남용

단순한 Grep 검색에 서브에이전트를 사용했던 시기가 있었다. 1회 검색에 3,000 토큰 이상을 소모했다.

개선: "10개 이상 파일 조사에만 서브에이전트" 규칙을 도입. 판단 플로우(제5장)를 습관화.

5위: CLAUDE.md 비대화

한때 CLAUDE.md가 500줄을 넘었다. 매 턴마다 수천 토큰의 고정 비용이 발생.

개선: 흑으로 이전할 수 있는 규칙을 분리. 이유를 1줄로 압축. 테이블 형식 적극 활용. 141줄까지 압축.

세션 길이와 토큰 효율

세션 시간과 토큰 효율의 관계를 관찰한 결과:

세션 시간	경향
30분 이하	시작 비용(시스템 프롬프트 로딩)의 비율이 높다. 너무 짧아 비효율
1~2시간	가장 균형이 좋다. <code>/compact</code> 1회로 충분
2~4시간	<code>/compact</code> 2~3회. 효율은 여전히 양호
4시간 이상	auto-compact 빈번 발생. 압축 후 맥락 손실로 재작업이 늘어남

최적 세션 시간은 2~3시간이었다. 그 이상은 세션을 나누는 편이 토탈 토큰 효율이 좋다.

/compact 타이밍과 효과

/compact 타이밍별 관찰 결과:

타이밍	컨텍스트 사용률	효과
50%에서 수동	너무 이름	압축 이점이 적다. 아직 여유가 있다
75%에서 수동	✅ 최적	충분한 정보를 압축. 세션이 오래 지속됨
93%에서 auto	너무 늦음	직전 턴 비용이 높다. 압축 품질도 낮다

75% 률은 체감뿐 아니라 데이터로도 뒷받침됐다.

흑 도입 전후 비교

9개의 흑을 도입하기 전후로 세션의 행동이 어떻게 달라졌는가:

지표	흑 도입 전	흑 도입 후
평균 세션 시간	1.5시간	2.5시간
auto-compact 빈도	2회/세션	0.5회/세션
재작업 (같은 작업 다시 하기)	3회/세션	1회/세션
대용량 파일 통째 읽기	5회/세션	0.2회/세션

흑의 효과 중 가장 컸던 것은 대용량 파일 통째 읽기 방지였다. 1회 통째 읽기가 수천 토큰이므로, 5회를 막는 것만으로도 세션이 수십 분 연장되는 계산이 나온다.

실측 데이터의 해석

같은 쿼터로 사용 가능한 세션 시간이 1.5시간 → 2.5시간으로 연장됐다. 이는 Max 플랜 고정 요금 대비 67% 더 많은 작업을 처리할 수 있음을 의미한다. 바꿔 말하면, 동일한 작업량을 달성하기 위한 토큰 소비가 약 40% 감소했다.

이 40%는 단일 시책이 아니라, 상태 관리 최적화(30% → 10% 미만), 대용량 파일 가드, CLAUDE.md 압축, auto-compact 빈도 75% 감소(2회 → 0.5회)—모든 시책을 조합한 총합 효과다.

개별 효과는 환경과 워크플로우에 따라 다르다. 본인 환경에서는 소비가 가장 많았던 "상태 관리의 과도한 업데이트"를 없앴 것만으로도 세션 시간이 눈에 띄게 늘어났지만, 애초에 그 문제가 없는 환경이라면 효과는 작을 것이다.

타이틀의 "절반으로 줄이기"는, 소비가 가장 많은 패턴을 가진 환경에서의 최대 효과를 표현한 것이다. 모든 환경에서 절반이 되는 것은 아니다. 단, 위의 표에 있는 시책을 아직 하나도 실시하지 않은 환경이라면, 30~50% 절감은 현실적으로 기대 가능한 수준이다.

월별 쿼터 추이

Max 플랜은 고정 요금이지만, 쿼터 소비 속도는 월별로 크게 달라졌다:

월	주요 활동	쿼터 여유
12월	초기 세팅, 학습 기간	여유 있음
1월	툴 개발 러시	주 중반에 쿼터 경고 발생
2월	아티클 작성 중심으로 전환	월말까지 지속
3월	혹 도입, 워크플로우 최적화	여유 생김
4월	최적화된 운영	안정

1월은 툴 개발로 대량의 파일을 읽고 쓰다 보니 소비가 가장 격렬했다. 2월 이후 아티클 작성(텍스트 생성 중심)으로 전환하면서 소비가 안정됐다.

3월의 혹 도입 이후에는 같은 종류의 작업이라도 소비가 감소했다.

1200시간에서 얻은 교훈

1200시간 운영을 통해 얻은 가장 중요한 교훈:

토큰 소비의 80%는 "읽기"에 있다 — 생성보다 파일·커맨드 출력의 읽기가 토큰을 더 많이 소모한다. 읽는 양을 줄이는 것이 가장 효과적이다.

혹은 "부탁"을 "강제"로 바꾼다 — CLAUDE.md의 규칙은 잊혀진다. 혹은 잊지 않는다. 도입 비용은 낮다.

세션 설계가 전체를 결정한다 — 1세션에 너무 많은 것을 담지 않고, 테마별로 나눈다. 이것만으로도 크게 개선된다.

수동 /compact는 투자다 — 일찍 압축하는 것은 얼핏 아까워 보이지만, 세션 전체로 보면 이익이다.

상태 관리 빈도를 줄인다 — "무엇을 하고 있는지" 기록하는 행위 자체가 토큰을 소모한다는 역설. 최소한으로 줄인다.

다음 장에서는 토큰 소비에 관한 트러블슈팅을 설명한다.

제8장 트러블슈팅 — 토큰이 갑자기 사라지는 원인과 대처법

버전 정보: 이 내용은 2026년 4월 기준 Claude Code(v2.1.x)에 기반합니다. GitHub Issue 번호는 원문 그대로 유지합니다.

"어제까지는 잘 쓰였는데 오늘은 쿼터가 순식간에 사라졌다" — GitHub Issue에서 가장 많이 보고되는 패턴이다.

이 장에서는 토큰 소비가 급증하는 원인과 그 대처법을 설명한다.

증상 1: 프롬프트 캐시 파괴

무슨 일이 일어나는가

Claude Code는 프롬프트 캐시라는 Anthropic API 기능을 사용하고 있다. 동일한 프롬프트(시스템 프롬프트, CLAUDE.md, 대화 기록의 앞부분)를 재사용함으로써 2번째 턴부터 토큰 소비를 대폭 줄인다.

이 캐시가 "파괴"되면 매 턴마다 프롬프트 전체가 새로 계산된다. 소비량이 수 배로 치솟는다.

원인

- **CLAUDE.md 변경:** 세션 중에 CLAUDE.md를 편집하면 캐시가 무효화된다
- **settings.json 변경:** 혹 추가·변경도 캐시를 파괴한다
- **MCP 서버 연결/해제:** 톨 정의가 바뀌므로 캐시가 무효화된다
- **git status 변경 (#47107):** Claude Code는 시스템 프롬프트에 현재의 git status를 포함한다. 파일을 1개만 편집해도 git status가 변경되어 캐시가 파괴된다. 코드를 작성할 때마다 캐시가 파괴되는 구조적인 문제
- **새 세션 시작 시 재생성 (#47098):** 새로운 세션에서는 skills·CLAUDE.md·설정 정보(약 6,500 토큰)의 캐시가 존재하지 않는다. 세션을 자주 전환하면 매번 6,500 토큰의 오버헤드가 발생한다
- **버전 업그레이드로 인한 시스템 프롬프트 팽창:** v2.1.104에서 시스템 프롬프트의 캐시 오버헤드가 94% 증가했다는 보고가 있다 (#47528, cold cache 기준 49.7K → 96.5K 토큰). v2.1.105에서는 cold start 시 오버헤드가 4% → 11%로 악화됐다 (#47659). 버전 업그레이드 후 토큰 소비가 늘었다면 Claude Code 자체의 변경이 원인일 수 있다
- **Anthropic 측의 인프라 변경:** 사용자 측에서는 제어할 수 없지만 드물게 발생한다

대처법

- 세션 중에 **CLAUDE.md**를 편집하지 않는다: 편집이 필요하면 새 세션을 시작한다
- **settings.json** 변경도 세션 시작 전에: 혹 추가·수정은 세션 밖에서 진행한다
- **git status**로 인한 캐시 파괴 방지: `export CLAUDE_CODE_DISABLE_GIT_INSTRUCTIONS=1` 로 비활성화할 수 있다. 단, git 관련 지시가 사라지므로 git을 많이 사용하는 워크플로우에서는 주의가 필요

하다

- 세션을 쉽게 전환하지 않는다: 1세션 = 1테마로, 테마가 완료될 때까지 유지한다. 전환할 때마다 6,500 토큰의 캐시 재생성 비용이 발생한다
- 갑자기 쿼터가 줄기 시작하면: 세션을 종료하고 새 세션을 시작한다. 캐시가 리셋된다

확인 방법

Claude Code UI 하단의 상태바에서 세션의 토큰 소비를 확인할 수 있다. Max 플랜 사용자의 경우, API 콘솔 (`console.anthropic.com`)이 아닌 Claude Code의 상태바와 Claude.ai 계정 설정이 확인처가 된다. 캐시가 유효할 때와 파괴됐을 때의 1턴당 소비에는 뚜렷한 차이가 있다.

증상 2: 쿼터가 갑자기 소진된다

무슨 일이 일어나는가

Max 플랜(월 \$200)에는 1일 사용 상한이 있다. 이 상한은 Anthropic의 서버 부하 상황에 따라 변동한다.

원인

- 실제로 너무 많이 사용한다: 장시간 자율 운영으로 소비가 누적됐다
- 서브에이전트 폭주: 하나의 서브에이전트가 대량의 톨 호출을 실행했다
- 병렬 서브에이전트의 파일 충돌: 여러 서브에이전트가 같은 파일을 동시에 읽고 쓰면 `"File modified since read"` 에러가 발생하고, 각 서브에이전트가 재시도를 반복한다. GitHub Issue #46968에서는 이 패턴으로 101,000 토큰 이상이 재시도 루프에 소모됐다는 보고가 있다
- **auto-compact**의 연쇄: compact 완료 → 바로 컨텍스트가 가득 참 → 또 compact 루프

대처법

- `token-budget-guard` 혹은 설정한다: 세션 단위 소비 상한을 설정하고 초과 전에 경고
- 서브에이전트 상한을 설정한다: `subagent-budget-guard` 로 5분간 실행 수를 제한
- 병렬 서브에이전트의 파일 범위를 분리한다: `"src/api/를 조사해줘"`, `"src/models/를 조사해줘"` 처럼 겹치지 않는 범위를 지정한다. 같은 파일을 건드리는 작업은 직렬로 실행
- **auto-compact**가 3회 이상 발동하면 세션을 종료한다: 그 이상 계속해도 효율이 나오지 않는다

증상 3: 세션 시작 직후인데 이미 무겁다

무슨 일이 일어나는가

`claude --resume` 으로 이전 세션을 재개하면 이전 컨텍스트가 그대로 복원된다. 이전에 75%를 사용하고 있었다면 재개 직후에도 75%에서 시작된다.

대처법

```
claude --resume
# 재개 직후:
# > /compact 이제부터 oo를 진행합니다
```

재개 후 바로 `/compact` 를 실행해 이전 맥락을 압축하고 나서 시작한다.

증상 4: 같은 작업을 반복하고 있다

무슨 일이 일어나는가

Claude Code가 같은 파일을 반복해서 읽거나, 같은 커맨드를 반복 실행하거나, 비슷한 검색을 반복하는 패턴.

원인

- `/compact` 후 맥락 손실: 압축으로 파일 내용을 잊어버려 다시 읽으려 한다
- 모호한 지시: "이 코드를 개선해줘"와 같은 지시로 반복 시행착오가 발생한다
- 혹 오설정: 혹이 톨을 차단 → 다른 방법을 시도 → 또 차단 루프

대처법

- 지시를 구체적으로 한다: "이 코드를 개선해줘" → "이 함수에 null 체크를 추가해줘"
- `/compact` 전에 다음 액션을 명기한다: `/compact` 다음은 `src/api/handler.ts` 수정
- 혹 로그를 확인한다: 혹이 의도하지 않은 차단을 하고 있지 않은지 확인

증상 5: 톨 결과 무단 잘림 (200K 톨 톨 상한)

무슨 일이 일어나는가

Claude Code에는 톨 결과 합계에 약 200K 톨 톨의 내부 예산 상한이 있다 (v2.1.x계에서 확인). 이 상한을 초과 하면 톨 결과가 무단으로 잘린다. 버전 업그레이드로 변경될 수 있다.

예를 들어 거대한 로그 파일을 Read로 읽거나, 큰 저장소에서 Grep을 실행한 경우 결과 일부가 잘려나가고, Claude는 불완전한 정보를 기반으로 판단한다. 잘림이 발생해도 사용자에게는 통지되지 않는다.

왜 톨 톨 문제인가

잘린 결과를 바탕으로 Claude가 잘못된 판단을 하면 재시도가 발생한다. 재시도마다 톨 톨이 추가 소모된다. "1 번에 끝날 작업이 3번 필요하게 됐다"는 패턴으로 보이지 않는 톨 톨 낭비가 발생한다.

대처법

- 큰 파일은 범위 지정으로 읽는다: Read의 `offset/limit` 파라미터로 필요한 부분만 읽는다
- Grep 결과를 줄인다: `head_limit` 파라미터로 결과 수를 제한한다
- 세션을 분할한다: 1개의 세션에서 대량의 파일 조작을 수행하지 않는다

증상 6: 가짜 레이트 리밋 (synthetic 엔트리)

무슨 일이 일어나는가

세션의 JSONL 로그에 `"model": "<synthetic>"` 이라는 엔트리가 혼입되는 경우가 있다. 이것은 API를 실제로 호출하지 않았는데 레이트 리밋 카운트에 산입되는 엔트리다.

ArkNill 씨의 65세션 분석에서 151개의 `<synthetic>` 엔트리가 발견됐다 (#41617).

확인 방법

```
# 세션 JSONL에서 <synthetic> 엔트리 검색
grep -c '"<synthetic>"' ~/.claude/projects/**/*.jsonl
```

영향

synthetic 엔트리가 많으면 실제 사용량 이상으로 레이트 리밋에 걸리기 쉬워진다. "그렇게 많이 쓰지 않았는데 쿼터가 줄어든다"는 경우 이것이 원인일 수 있다.

대처법

2026년 4월 기준으로 Anthropic 측에서 미수정 상태. 사용자 측의 회피책은 없다. 이 문제를 인지하고, 쿼터 소비가 이상하다고 느껴지면 synthetic 엔트리 수를 확인하고 지원팀에 보고하는 것을 권장한다.

증상 7: "아무것도 안 했는데 토큰이 줄어든다"

무슨 일이 일어나는가

사용자가 조작하지 않아도 다음과 같은 이유로 토큰이 소비된다:

- **다른 터미널의 세션:** 여러 터미널에서 Claude Code를 열고 있는 경우, 사용하지 않는 세션에서도 auto-compact, retro, 혹은 처리가 백그라운드에서 실행된다. 어떤 사례에서는 백그라운드 세션이 쿼터의 78%를 소비하고 있었다. 사용하지 않는 세션은 `/exit` 로 명시적으로 종료할 것
- **자율 운영 중 톨 호출:** 방치해두는 동안 파일 읽기·쓰기를 반복하고 있다
- **MCP 서버와의 통신:** MCP 서버가 연결된 경우 톨 정의 재전송이 발생하는 경우가 있다
- **watchdog이나 nudge에 대한 응답:** 자율 운영에서 위치독이 정기적으로 확인을 실행하고 응답할 때마다 토큰을 소비한다

대처법

- **자율 운영 로그를 확인한다:** 무엇을 하고 있었는지 확인한 후 "아무것도 안 했다"고 판단한다
- **불필요한 MCP 서버를 연결 해제한다:** 사용하지 않는 MCP 서버는 `settings.json` 에서 제거한다
- `session-token-counter` 혹은 설정한다: 세션 단위 톨 호출 횟수를 모니터링한다

증상별 대처 빠른 참조표

증상	먼저 확인할 것	대처
갑자기 쿼터 급감	상태바에서 소비 확인	캐시 파괴 가능성 → 세션 재시작
툴 결과가 불완전	큰 파일을 읽은 직후인지	범위 지정으로 다시 읽기 (200K 상한)
안 썼는데 쿼터 감소	synthetic 엔트리 수	<code>grep -c '"<synthetic>"' *.jsonl</code> 로 확인
시작 직후부터 무거움	--resume으로 재개했는지	재개 직후 /compact
같은 작업의 반복	흑 로그 확인	지시 구체화, 흑 오설정 수정
아무것도 안 했는데 감소	자율 운영 로그	툴 호출 횟수 모니터링 흑
버전 업데이트 후 증가	<code>claude --version</code> 확인	#47528/#47659 참고. 구버전으로 되돌리거나 수정 대기
세션이 짧다	CLAUDE.md 줄 수	100줄 이내로 압축
허가 다이얼로그가 많다	permission 설정	안전한 툴을 allow + 흑으로 방어
MCP 툴을 찾지 못한다	MCP 서버 수 확인	10개 이하로 줄이기, 불필요한 서버 연결 해제

증상 8: 퍼미션 요청의 토큰 오버헤드

무슨 일이 일어나는가

Claude Code가 툴을 사용할 때마다 퍼미션 확인이 표시된다. 사용자가 승인할 때까지 대기하고, 승인 후 실행 — 이 반복 자체가 토큰을 소비한다.

구체적으로는 Claude가 bash 커맨드나 파일 편집을 제안할 때, 커맨드 전체나 코드 변경분을 컨텍스트에 포함한 채 퍼미션 프롬프트를 생성한다. 사용자가 승인하면 같은 코드가 다시 컨텍스트에 올라와 실행된다. 10회의 퍼미션 확인이 있으면 코드 조각이 20회 컨텍스트에 나타나는 계산이 된다.

GitHub Issue #41617의 2026년 4월 코멘트에서 이 문제가 구체적으로 보고되고 있다. "blanket permission(포괄적인 허가)을 부여하는 수단이 없어서 비슷한 작업마다 허가를 요청받고, 그때마다 토큰이 소비된다"는 지적이다.

대처법

퍼미션 설정으로 알려진 안전한 작업을 자동 허가한다:

```
{
  "permissions": {
    "allow": [
      "Bash(npm test:*)",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Read",
      "Glob",
      "Grep"
    ]
  }
}
```

읽기 계열 톨(Read, Glob, Grep)과 부작용이 적은 커맨드(test, status, diff)를 자동 허가로 설정하면 퍼미션 확인 횟수가 대폭 줄어든다.

- **--permission-mode** 를 활용한다: 신뢰할 수 있는 프로젝트에서는 `plan` 모드로 실행하고, 계획 승인 후에는 톨 호출을 자동 실행시킨다
- **PreToolUse** 훅으로 안전을 담보한다: 퍼미션을 완화하는 대신 훅으로 위험한 작업을 차단한다. 이것이 `cc-safe-setup` 의 접근법이다 — 667개 이상의 훅이 안전장치로 기능하므로 퍼미션 설정을 완화해도 위험을 억제할 수 있다

⚠ **v2.1.101 이후 변경 사항 (2026년 4월, 버전은 변경될 수 있음):** 씹킹 요약(thinking summaries)이 기본적으로 비표시로 변경됐다. 활성화하려면 `settings.json` 에 `"showThinkingSummaries": true` 를 추가한다. 씹킹 요약은 컨텍스트에 포함되므로 비활성화하면 토큰 소비가 미세하게 줄어든다. 반면, 요약이 보이지 않으면 Claude Code가 "무엇을 생각하고 있는지"를 파악하기 어려워지고, 불필요한 작업을 멈추는 타이밍이 늦어지는 위험도 있다.

예방책: 트러블을 일으키지 않기 위한 설정

다음 훅을 최소한으로 설정해두면 많은 트러블을 사전에 방지할 수 있다:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/large-read-guard.sh"}]
      },
      {
        "matcher": "Agent",
        "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/subagent-budget-guard.sh"}]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "",
```

```

    "hooks": [{ "type": "command", "command": "bash ~/.claude/hooks/token-
budget-guard.sh" }]
  },
],
"PreCompact": [
  {
    "matcher": "",
    "hooks": [{ "type": "command", "command": "bash pre-compact-
checkpoint.sh" }]
  }
]
}
}
}

```

이 4가지만으로:

- 큰 파일의 통째 읽기를 경고
- 서브에이전트 남용을 제한
- 토큰 예산 초과를 경고
- /compact 전에 작업을 자동 저장

증상 9: 혹은 설정해도 우회된다

무슨 일이 일어나는가

2026년 4월, 보안 연구팀(Adversa AI)이 Claude Code의 deny rules 우회 취약점을 발견했다. 50개 이상의 서브커맨드를 연결하면 모든 deny rules가 무효화된다.

내부적으로 `MAX_SUBCOMMANDS_FOR_SECURITY_CHECK = 50` 이라는 하드코딩된 상한이 있으며, 50을 초과하면 확인 없이 실행된다.

토큰에 대한 영향

이 취약점이 악용되면:

- 차단되어야 할 파괴적 커맨드가 실행된다
- 의도하지 않은 재시도 루프가 발생해 토큰이 급소비된다
- 자율 운영 중 제어가 듣지 않아 하룻밤 만에 £140 소비된 사례도 있다 (#47049)

대처법

deny rules 대신 PreToolUse 혹은 방어한다. 혹은 서브커맨드 수의 제한을 받지 않는다.

```
# subcommand-chain-guard.sh
THRESHOLD=${CC_SUBCOMMAND_LIMIT:-20}
INPUT=$(cat)
CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty' 2>/dev/null)
[ -z "$CMD" ] && exit 0

SUBCOMMAND_COUNT=$(echo "$CMD" | tr ';' '\n' | tr '&' '\n' | tr '|' '\n' |
grep -c '^[^ ]' 2>/dev/null || echo 1)

if [ "$SUBCOMMAND_COUNT" -gt "$THRESHOLD" ]; then
    echo "BLOCKED: $SUBCOMMAND_COUNT subcommands (limit: $THRESHOLD)." >&2
    exit 2
fi
exit 0
```

cc-safe-setup 에 포함되어 있다: `npx cc-safe-setup --install-example subcommand-chain-guard`

증상 10: MCP 툴의 eager 로드로 인한 컨텍스트 압박

무슨 일이 일어나는가

MCP 서버를 다수 연결한 환경에서 ToolSearch 가 `result=false` 를 반환하고 툴을 찾지 못하는 현상이 보고되고 있다 (#47645). 21개의 MCP 서버를 연결한 사례에서는 툴 정의가 eagerly(일괄·즉시)로 로드되어, 세션 시작 시점에 컨텍스트의 14%가 MCP 툴 정의만으로 소비됐다.

토큰에 대한 영향

툴 정의는 시스템 프롬프트의 일부로 캐시되지만, MCP 서버의 연결 상태가 바뀔 때마다 캐시가 파괴된다 (증상 1 참고). 게다가 컨텍스트의 14%가 툴 정의로 차지되면 실제 작업에 사용할 수 있는 용량이 줄어들고, auto-compact 발동이 빨라지는 악순환에 빠진다.

대처법

- 사용하지 않는 MCP 서버를 연결 해제한다: `settings.json` 에서 불필요한 MCP 서버를 삭제하고 필요할 때만 연결한다
- MCP 서버 수를 10개 이하로 유지한다: 10개를 초과하면 툴 정의의 오버헤드가 뚜렷해진다
- ToolSearch가 실패하는 경우: MCP 서버 수를 줄이고 세션을 재시작한다

증상 11: 몇 개의 프롬프트로 쿼터 절반이 사라지는 이상 소비

무슨 일이 일어나는가

Pro 플랜 사용자가 Sonnet과 Opus에 각 1회씩(총 2 프롬프트)을 보냈을 뿐인데 월간 사용량의 **49%**를 소비한 사례가 보고되어 있다 (#47587). \$200 플랜 사용자에서도 3시간에 주간 상한에 도달하는 케이스가 코멘트에서 확인되고 있다.

왜 발생하는가

- **세션 상태의 캐시 불일치**: 플랜 업그레이드 직후 크레딧이 반영되지 않는 케이스 (#47641)가 있다. API 대시보드에서는 0% 사용인데 429 에러가 반환된다
- **컨텍스트 팽창과의 복합**: 증상 1~3(캐시 파괴, 버전 기인 비대화, 컨텍스트 팽창)이 동시에 발생하면 1 프롬프트당 토큰 소비가 통상의 수 배가 된다
- **서버 측 계량 버그**: 사용자 측에서 제어할 수 없지만 드물게 발생한다

대처법

- `/cost` 로 매 턴 확인하는 습관을 들인다: 이상한 증가를 발견하면 세션을 종료한다
- `token-budget-guard` 혹은 설정한다: 1턴당 토큰 소비에 상한을 설정한다 (제4장 참고)
- 플랜 업그레이드 후에는 세션을 재시작한다: 같은 세션 내에서는 크레딧이 반영되지 않는 경우가 있다
- **API 대시보드와 대조한다**: Claude Code 측의 표시와 API 대시보드의 수치가 괴리된 경우 Anthropic 지원팀에 보고한다

증상 12: Opus의 씹킹 루프로 인한 무한 토큰 소비

무슨 일이 일어나는가

Opus 4.6이 thinking(내부 사고)의 루프에 빠져 응답을 생성하지 않고 토큰만 소비하는 현상이 보고되어 있다 (#47602, 관련: #24585, #37023). 사용자 관점에서는 "생각 중" 상태가 수 분간 지속되고, 쿼터만 줄어간다.

토큰에 대한 영향

thinking 토큰은 출력 토큰으로 계상된다. Opus 4.6의 thinking은 1회에 수천~수만 토큰을 소비하는 경우가 있으며, 루프하면 단시간에 대량의 쿼터가 손실된다.

대처법

- **응답이 30초 이상 멈추면 Esc/Ctrl+C로 중단한다**: 조기 중단으로 토큰 낭비를 최소화
- **작업을 작게 분할한다**: 복잡한 지시를 한꺼번에 1개의 프롬프트에 넣으면 thinking이 길어지는 경향이 있다. 구체적이고 작은 지시로 나눈다
- **Sonnet으로 전환해서 테스트한다**: thinking 루프는 Opus 고유의 문제다. 같은 지시가 Sonnet에서는 정상적으로 동작하는 경우, 지시의 입도가 아닌 모델 측의 문제다
- **/clear하고 재시도한다**: 컨텍스트의 누적이 thinking의 깊이에 영향을 주는 경우가 있다

보충: 캐시 TTL 실측 데이터

증상 1에서 언급한 캐시 TTL에 관해, 1,610건의 API 콜 분량의 텔레메트리에 기반한 정량 분석이 공개되어 있다 (#47425). 이 분석에 따르면 89.6%의 API 콜이 1시간 캐시 티어를 사용하고 있으며, 5분 아이들 초과 시 캐시가 전체 재구축되는 것의 영향이 정량화됐다. 현재 5분과 1시간 사이에 중간 티어(15분, 30분 등)를 설정해야 하는지에 대한 논의가 진행 중이다. 5분 초과로 인한 전체 재구축을 피하고 싶은 경우, 세션을 일정 간격으로 가벼운 작업(`/cost` 확인 등)으로 활성 상태로 유지하는 것이 효과적이다.

다음 장에서는 바로 사용할 수 있는 CLAUDE.md, hooks, settings.json 템플릿을 수록한다.

제9장 설정 템플릿 모음 — 복붙으로 바로 쓰는 CLAUDE.md · hooks · settings.json

이 장은 각 장에서 해설한 내용을 "그대로 바로 쓸 수 있는" 형태로 정리한 템플릿 모음이다. 복붙해서 자신의 환경에 맞게 조정해 사용하길 바란다.

템플릿 1: 토큰 효율 중시 CLAUDE.md

```
# 프로젝트 지시서

## 기본 방침
- 파일 읽기는 limit 파라미터로 필요한 부분만 읽는다
- 1세션 = 1테마. 테마가 바뀌면 새 세션을 시작한다
- 혹은 차단한 작업은 재시도하지 않는다

## 코딩 규약
| 상황 | 판단 |
|-----|-----|
| 기술 선택 | 가장 대중적인 것 |
| 불명확한 요구사항 | 가장 단순한 해석 |
| 보안 | 항상 안전한 쪽 |

## 커밋
✓ fix: prevent duplicate API calls on rapid button clicks
✗ fix bug

## 금지
혹이 차단하는 작업(rm -rf, git reset --hard 등)은 재시도하지 않는다.
```

포인트: 약 30줄. 제2장의 5패턴(허용 목록, 구체적 예시, 이유 1줄, 테이블, 혹은 분리)을 모두 적용.

템플릿 2: 토큰 절약 혹은 세트 (settings.json)

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/large-read-guard.sh", "timeout": 5000}]
      },
      {
        "matcher": "Read",
        "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/read-budget-guard.sh", "timeout": 5000}]
      }
    ]
  }
}
```

```

    },
    {
      "matcher": "Agent",
      "hooks": [{"type": "command", "command": "bash
~/.claude/hooks/subagent-budget-guard.sh", "timeout": 5000}]
    },
    {
      "matcher": "Read",
      "hooks": [{"type": "command", "command": "bash
~/.claude/hooks/duplicate-read-detector.sh", "timeout": 5000}]
    },
    {
      "matcher": "Bash",
      "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/bash-
output-guard.sh", "timeout": 5000}]
    }
  ],
  "PostToolUse": [
    {
      "matcher": "",
      "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/token-
budget-guard.sh", "timeout": 5000}]
    },
    {
      "matcher": "",
      "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/token-
spike-alert.sh", "timeout": 5000}]
    }
  ],
  "PreCompact": [
    {
      "matcher": "",
      "hooks": [{"type": "command", "command": "bash ~/.claude/hooks/pre-
compact-checkpoint.sh", "timeout": 10000}]
    }
  ],
  "SessionStart": [
    {
      "matcher": "",
      "hooks": [{"type": "command", "command": "bash
~/.claude/hooks/session-budget-alert.sh", "timeout": 5000}]
    }
  ]
}

```

포인트: 제4장에서 소개한 9개의 훅을 모두 설정. timeout은 각 훅 5초(checkpoint만 10초).

템플릿 3: 훅 파일 일체

다음 파일들을 `~/.claude/hooks/` 에 배치한다.

large-read-guard.sh

```
#!/bin/bash
# 100KB 초과 파일 읽기 경고
input=$(cat)
file_path=$(echo "$input" | jq -r '.tool_input.file_path // empty')

if [ -n "$file_path" ] && [ -f "$file_path" ]; then
    size=$(stat -c%s "$file_path" 2>/dev/null || stat -f%z "$file_path" 2>/dev/null)
    threshold=$(( ${CC_LARGE_READ_KB:-100} * 1024 ) )
    if [ "${size:-0}" -gt "$threshold" ] 2>/dev/null; then
        echo "⚠️ ${file_path} 는 $((size / 1024))KB. limit 파라미터로 필요한 부분만 읽어주세요."
        exit 0 # 경고만 (차단하려면 exit 2)
    fi
fi
```

read-budget-guard.sh

```
#!/bin/bash
# 세션 중 읽기 파일 수 제한
budget=${CC_READ_BUDGET:-100}
warn=${CC_READ_WARN:-50}
tracker="/tmp/cc-read-budget-$PPID"

input=$(cat)
file_path=$(echo "$input" | jq -r '.tool_input.file_path // empty')
[ -z "$file_path" ] && exit 0

if [ -f "$tracker" ]; then
    grep -qF "$file_path" "$tracker" || echo "$file_path" >> "$tracker"
else
    echo "$file_path" > "$tracker"
fi

count=$(wc -l < "$tracker")
if [ "$count" -ge "$budget" ]; then
    echo "🚫 읽기 ${budget}건 도달. Glob/Grep으로 범위를 좁혀 주세요."
    exit 2 # exit 2 = 차단
elif [ "$count" -ge "$warn" ]; then
    echo "⚠️ 읽기 ${count}/${budget}건."
fi
```

duplicate-read-detector.sh

```
#!/bin/bash
# 동일 파일 재읽기 경고
tracker="/tmp/cc-read-history-$$PPID"
input=$(cat)
file_path=$(echo "$input" | jq -r '.tool_input.file_path // empty')

if [ -n "$file_path" ] && [ -f "$tracker" ]; then
    if grep -qF "$file_path" "$tracker"; then
        echo "⚠️ ${file_path} 는 이미 읽은 파일입니다. 재읽기가 불필요할 수도 있습니다."
    fi
fi

echo "$file_path" >> "$tracker"
```

token-budget-guard.sh

```
#!/bin/bash
# 토큰 예산 관리
budget_warn=${CC_TOKEN_BUDGET:-50000}
budget_block=${CC_TOKEN_BLOCK:-100000}
tracker="/tmp/cc-token-budget-$$PPID"

input=$(cat)
tokens=$(( ${#input} / 4 ))

total=0
[ -f "$tracker" ] && total=$(cat "$tracker")
total=$((total + tokens))
echo "$total" > "$tracker"

if [ "$total" -ge "$budget_block" ]; then
    echo "🔴 추정 ${total} 토큰 소비. /compact를 실행해 주세요."
    exit 2 # exit 2 = 차단
elif [ "$total" -ge "$budget_warn" ]; then
    echo "⚠️ 추정 토큰: ${total}/${budget_block}"
fi
```

subagent-budget-guard.sh

```
#!/bin/bash
# 서버에이전트 수 제한
max=${CC_MAX_SUBAGENTS:-5}
tracker="/tmp/cc-subagent-$$PPID"
now=$(date +%s)
echo "$now" >> "$tracker"

cutoff=$((now - 300))
[ -f "$tracker" ] && count=$(awk -v c="$cutoff" ' $1>=c ' "$tracker" | wc -l)
|| count=1
if [ "$count" -gt "$max" ]; then
    echo "🚫 5분 동안 서버에이전트 ${count}개 (상한 ${max}개). 직접 Grep/Glob을 사용해 주
세요."
    exit 2 # exit 2 = 차단
fi
```

token-spike-alert.sh

```
#!/bin/bash
# 톨 호출 빈도 모니터링
tracker="/tmp/cc-spike-$$PPID"
now=$(date +%s)
echo "$now" >> "$tracker"

cutoff=$((now - 30))
[ -f "$tracker" ] && {
    awk -v c="$cutoff" ' $1>=c ' "$tracker" > "${tracker}.tmp"
    mv "${tracker}.tmp" "$tracker"
    count=$(wc -l < "$tracker")
    [ "$count" -ge 15 ] && echo "⚠️ 30초 동안 ${count}회 톨 호출. 폭주 가능성이 있습니
다."
}
```

pre-compact-checkpoint.sh

```
#!/bin/bash
# /compact 전 git commit
cd "$(git rev-parse --show-toplevel 2>/dev/null)" || exit 0
if [ -n "$(git status --porcelain)" ]; then
    git add -A # .gitignore로 .env나 기밀 파일을 제외해 둘 것
    # --no-verify: 체크포인트 전용이므로 pre-commit hooks 건너뛰
    git commit -m "checkpoint: pre-compact $(date +%Y%m%d-%H%M%S)" --no-verify
    2>/dev/null
    echo "✅ 체크포인트 저장 완료."
fi
```

bash-output-guard.sh

```
#!/bin/bash
# 대출력 커맨드 경고
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command // empty')
if echo "$cmd" | grep -qE "git log|find / |cat |strings |hexdump"; then
    echo "$cmd" | grep -qE "\\| head\\|\\| tail\\|\\| grep|-n [0-9]" || \
    echo "⚠️ 출력이 커질 수 있습니다. | head -50 을 검토해 주세요."
fi
```

session-budget-alert.sh

```
#!/bin/bash
# 세션 시작 시 예산 표시
state="/tmp/cc-daily-budget"
today=$(date +%Y-%m-%d)
if [ -f "$state" ] && [ "$(head -1 "$state")" = "$today" ]; then
    n=$(tail -1 "$state")
    echo "$today" > "$state" && echo "$((n+1))" >> "$state"
    echo "📊 오늘 $((n+1))번째 세션입니다."
else
    echo "$today" > "$state" && echo "1" >> "$state"
    echo "📊 오늘 첫 번째 세션입니다."
fi
```

설치 절차

```
# 1. hooks 디렉토리 생성
mkdir -p ~/.claude/hooks

# 2. 위의 hooks 파일들을 배치
# (각 파일을 ~/.claude/hooks/ 에 복사)

# 3. 실행 권한 부여
chmod +x ~/.claude/hooks/*.sh

# 4. settings.json에 템플릿 2의 내용을 추가
# ~/.claude/settings.json 을 편집

# 5. 동작 확인
claude
# > 세션 시작 시 "오늘 첫 번째 세션입니다"가 표시되면 OK
```

cc-safe-setup을 사용하는 경우

위의 hooks를 하나씩 설정하는 대신 `cc-safe-setup`을 사용하면 커맨드 하나로 설치할 수 있다:


```
npx cc-safe-setup --install-example token-budget-guard
npx cc-safe-setup --install-example large-read-guard
npx cc-safe-setup --install-example subagent-budget-guard
# ... 다른 혹도 동일하게
```

cc-safe-setup에는 667개 이상의 혹이 준비되어 있다. 본서에서 소개한 9개의 혹 중 large-read-guard, read-budget-guard, token-budget-guard, token-spike-alert, subagent-budget-guard, pre-compact-checkpoint, session-budget-alert의 7개는 cc-safe-setup에 포함되어 있다. duplicate-read-detector와 bash-output-guard는 본서 오리지널이므로 위의 템플릿 3에서 복사해 사용한다.

환경 변수로 커스터마이징

혹의 임계값은 환경 변수로 조정할 수 있다. ~/.claude/settings.json의 env 섹션 또는 .bashrc / .zshrc에 추가한다:

```
# 대용량 파일 임계값 (KB)
export CC_LARGE_READ_KB=200      # 기본값: 100

# 읽기 파일 수 상한
export CC_READ_BUDGET=150        # 기본값: 100
export CC_READ_WARN=120         # 기본값: 50

# 토큰 예산
export CC_TOKEN_BUDGET=80000     # 경고 임계값. 기본값: 50000
export CC_TOKEN_BLOCK=150000    # 차단 임계값. 기본값: 100000

# 서브에이전트 상한
export CC_MAX_SUBAGENTS=3       # 기본값: 5
```

자신의 사용 방식에 맞게 조정하길 바란다. 처음에는 기본값으로 시작해서 1~2주 사용해본 후에 조정하는 것을 권장한다.

마치며

템플릿 4: .claudeignore (불필요 파일 제외)

```
# .claudeignore - Claude Code가 읽지 않을 파일을 지정
# .gitignore와 동일한 문법. 프로젝트 루트에 배치

# 빌드 산출물
dist/
build/
.next/
out/

# 의존성 (대량의 파일)
```

```
node_modules/  
vendor/  
.venv/  
__pycache__/  
  
# 로그·임시 파일  
*.log  
*.tmp  
coverage/  
  
# 대용량 데이터 파일  
*.sql  
*.csv  
*.sqlite  
fixtures/  
  
# 생성된 파일  
*.min.js  
*.bundle.js  
*.map
```

포인트: `.gitignore`로 이미 제외한 항목은 `.claudeignore`에 중복해서 쓸 필요가 없다. Claude 고유의 제외 항목(테스트 픽스처, 대용량 문서 폴더, SQL 덤프 등)을 추가한다.

이 책은 800시간의 자율 운영에서 배운 토큰 절약 테크닉을 체계화한 것이다.

요점을 3가지로 정리하면:

1. 읽는 양을 줄인다 — 대용량 파일 제한, 중복 읽기 방지, 서브에이전트의 적절한 사용
2. 컨텍스트를 관리한다 — 75% 룰, 수동 /compact, 세션 분할
3. 흑으로 강제한다 — 규칙을 "부탁"에서 "자동 차단"으로 바꾼다

토큰 소비는 "어림짐작"으로 개선할 수 있는 것이 아니다. 구조로 제어한다. 이 책의 템플릿이 그 첫걸음이 되길 바란다.

저자: 김재우 jaewoo@mdrules.dev